

Table of Contents

CSE 221: Reading Schedule

9/25 Intro

- Course Structure

9/30 History

1. [THE System](#)

- sequential processes → ordering what is running, syncing with semaphores
- memory storage using *segment* + *page number* instead of physical location
- layers 0 → 5 : scheduler, segment controller, message interpreter, peripheral management, programs, operator

2. [RC 4000 \(Nucleus\)](#)

- Kernel – IPC w message passing, proc create/start/stop/remove, preemption, exceptions, scheduling
- OS on top – memory management, parent_procs manage children procs
- internal proc = stack+code+heap, external proc = hardware device wrapper
- **Economy of abstraction** → small amount of reused abstractions

10/2 Protection

3. [TENEX](#)

- Virtual memory – BBN pager, multilevel page table, TLB, shared pages w/ C-O-W, multi_proc, file sys w/o nested dirs, exec shell, backward binaries w/ JSYS wrapper

4. [UNIX](#)

- ordinary, directories, special files, multi-user system, shell made dev easier,
- used fd abstraction for directories, flags for being used, I/O = special file
- multics = list of users, unix = perm bits for users rwx (capabilities list)
- moved shell to user-level, stdin: fd=0, stdout: fd=1, piping in and out, fork, exec
 - [fork\(\) Discussion \(Baumann/HotOS'19\)](#) (*optional*)

10/6 Protection

5. [HYDRA](#)

- LNS → objects + rights = capabilities list, checked on every prod call
- open(C_dir, name)→kernel→ dir-LNS→check perms→return C_file to caller LNS
- read(C_file, buf)→kernel→file-LNS (with C_file)→check rights→do I/O

6. [Protection Domains \(Lampson\)](#)

- access control list (per-object perms), separated users vs. files (either can be domain)

- memory bounds checking physical registers
- `open(file)`→kernel check access matrix→allow/deny→return fd/name→do I/O

10/9 Protection

7. Multics

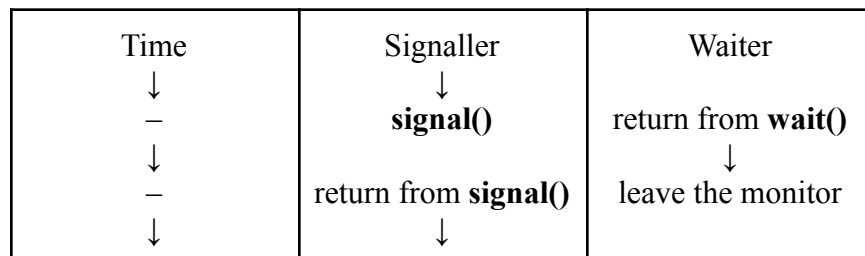
- minimize privilege, whitelist, check every action (at runtime) against ACL,
- segment `v_mem`: `procs=code,stack,file...`, `OS=fs, IPC,...` `v_addr = segment# + offset`
- 8 rings = layers → gates = (`syscall`) | `coreOS`→`drivers`→`fs`→`libs`→`apps`→ `users`

8. Singularity

- One address space, one language, one ptr address access, very every access w CBC
- `write(fileEP, buf)`→SIP runtime→`Write(msg, buf)` on file channel→kernel enforces CBC on file →file-system SIP checks capability/rights→does I/O

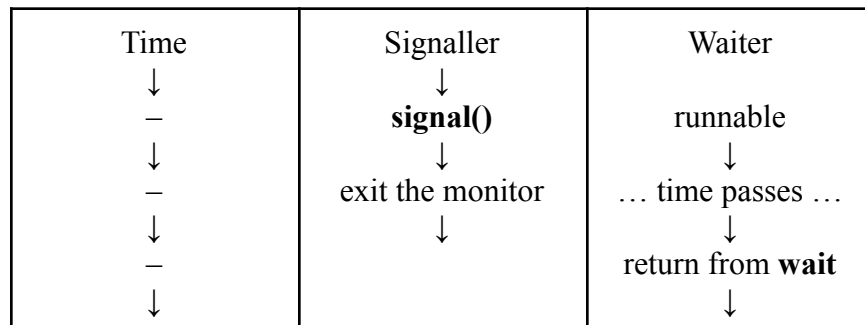
10/14 Synchronization

9. Monitors (Hoare)



■

10. Mesa Monitors



■

10/16 Virtual Memory

11. VAX/VMS

- 3 layers: `P0`, `P1`=user-space, `10`=kernel-space, linear page table entries
- page caching to free-lists, TLB split user/kernel, cluster I/O batch read, collect writes
- `addr_space`: `0`→`n` = `kernel[res→sys]` → `user[P1:prog/heap` → `P0:ctrl/runtime/buf]`

12. Mach VM

- `addr_map = 0` : code ↔ data ↔ heap ↔ file ↔ stack : 2^{32} ; holds prot + addr range
- `pmap` = HW per-task mapping (`v_addr`→`p_addr`), rebuildable w Mach res PTE

10/21 Distributed

13. Sprite

- multiprocessing, network OS, net connect machines, RPC procs between comps
- Cached files on server and client, using version numbers with timestamps

14. LegoOS

- Splitkernel → pComp, mComp, sComp, split to allow individual scaling,
- monitors per component for syncing. pComp has 4GB exCache for mem efficiency
- mComp manages 1GB vRegions; VMA tree for virtual addr mapping to vRegion,

10/23 Extending the OS

15. Microkernel (L4)

- Minimize kernel, keep only IPC, addr spaces, & threads in kernel-level
- each apps gets libc, use lib calls to req kernel for resources, IPC to communicate
- user-pages→kernel-map (`v_addr:pfn`)→HW-page-tables (MMU/TLB)

16. Exokernel

- Push HW to user-level, app + libOS; libOS→exokernel to manage HW
- `open(file)` → libFS → EXO=disk_blocks+bindings→check perms

10/28 Virtualization

17. VM/370

■

18. Xen

10/30 Virtualization

19. Containers (Soltesz/Eurosys'07)

20. Core Slicing (Zhou/OSDI'23)

11/4 Scheduling

21. Scheduler Activations

- Arachne (Qin/OSDI'18) (*optional*)

22. Linux Scheduler (Lozi/Eurosys'16)

11/6 Scalability

23. [RCU \(McKenney/OSR'20\)](#)

24. [Linux Scalability](#)

- [Commutativity Rule \(Clements/SOSP'13\)](#) (*optional*)

11/11 Holiday: Veteran's Day

11/13 Networking

25. [Remote Procedure Call \(Birrell & Nelson\)](#)

26. [Receive Livelock](#)

11/18 Networking

27. [IX Dataplane](#)

28. [Snap \(Microkernel Networking\)](#)

11/20 File Systems

29. [Fast File System \(FFS\)](#)

30. [Log-Structured File System \(LFS\)](#)

11/25 No class

11/27 Holiday: Thanksgiving

12/2 File Systems

31. [Soft Updates](#)

- [SquirrelFS \(Rust Consistency\)](#) (*optional*)

32. [SplitFS \(Persistent Memory\)](#)

12/4 File Systems

33. [Google File System \(GFS\)](#)

- [Crash Verification \(Sigurbjarnarson/OSDI'16\)](#) (*optional*)

Intro 9/25

CSE 221: Graduate Operating Systems

Instructor: Amy Ousterhout Wednesday 1:00 – 2:00pm @ CSE 3130

TA: Tianyi Shan Monday 1:00 – 2:00pm @ CSE 2236

- What **problem** does the paper address?
- How is it **different** from previous work, if any?
- What is the **approach** used to solve the problem?
- How does the paper **support** or otherwise justify its arguments and conclusions?
- Was the paper, in your judgement, **successful** in addressing the problem?

Why grad OS?

- Broadens OS horizons
- Learn not just what but why

What will I get out of this course?

- Principles for OS design
- Experience reading research papers
- Comfort during discussing research material
- Experience measuring systems performance (**final project**)

Course Structure

Research Papers

- 33 papers
- Grouped into topics
 - Protection
 - Synchronization
 - Virtual memory
 - File systems

Discussions

- Interactive and discussion based class
 - (no slides) - uses socratic method
- Participation is key!
- Paper reading approach
 - Considered authors goals,

- Compare diff papers in same subject
- What defense do they give for their work
- Did they complete their goals

Homeworks

- 3 assignments
- Consider the reasoning behind design choices

Project

- System measurement project
 - Choose a system
 - Measure aspects of performance
 - access memory vs cache / perform syscall vs pagecall
 - Ensure consistency
- 1 – 3 students allowed
- Gain experience measuring software performance
- Intuition about overheads - good context for papers
- Implementation experiences for benchmarking
- Technical writing
 - Final report: methodologies, results and analysis
 - Two checkpoints
- In-person check in after first two check points

Final Exam

- No midterm
- 1 cheat sheet ; 8.5 x 11 allowed

Miscellaneous

- Papers are not necessarily definitive truth
- Assessing papers is a skill developed over time
- Course website: <https://cseweb.ucsd.edu/classes/fa25/cse221-a/>
 - ☐ Canvas
 - ☐ Piazza
 - ☐ Gradescope

Generative AI policy

- ✓ Search engine like usage is okay - what is this vs that, or review this topic
- ✗ Summarize research papers

- ✗ Answer homework questions
- ✗ Copy and paste any code
- ✗ Writing your report

Enrollment

- ☒ ~~Complete undergrad OS course (CSE 120)~~
- ☐ Submit an EASy request

Class Notes

What is an Operating System?

- Interface **between** hardware and applications
 - Managing hardware **resources**
 - Isolation, security, protection
 - manager/**starter** of all running on a machine
 - **abstractions** of hardware resources
 - Illusions to applications
 - virtualize memory
 - ex. Isolation of processes
 - Handles interrupts and exceptions
 - Define policies & enforce them
- | Applications | [] [] [] ↗ abstractions

| Operating System |

| Hardware | [] [] [] ↘ management

Windowing Environment - part of the OS?

- No – relies on syscalls to work
- Depends on the OS – on some platforms you can choose the windowing environment
 - For windows ; used to be user level, now part of OS
 - For linux ; user-level

Web Browser

- No – there are many options for web browsers
- No – User - level program
- No – Sits above OS to interact between user and network
- Yes – some browsers are built into the OS (safari, edge)
 - Microsoft argued it was part of OS, forced users to use internet explorer
- Typically not considered part of the OS today

Network Stack

- Yes - use syscalls to access the network

- Device drivers ? – loaded into the OS , but technically external code
- Not always – might implement stack at user level for performance
- It depends on the layer (OSI model) →
 - NIC, ethernet, IP, and TCP are OS Level
 - HTTP, presentation, and application are User Level

Memory Allocation

- Yes – management responsibilities for physical memory
- Yes – users only deal with virtual memory abstraction
- No – some control of memory from user level (stack or heap) ← but these are virtual
- Yes – OS provides interfaces to request memory, but OS is in charge
- Both – depends: ex. garbage collection & malloc/free are user-level
- Yes – OS manages physical resources
- Split between OS and userspace
 - [App] → malloc → [libc] user-level
 - mmap/sbrk OS level

How different are different OSes?

- Similar – all managing hardware resources
- Depends – some are more configurable than others
- Different – based on settings ; mobile devices vs airplane controller vs RTOC
- Different – support for low-level hardware
 - x86 vs ARM
- Overall they carry similar goals, but have different approaches

History - 9/30

AI usage acknowledgement [ChatGPT history](#)

E. W. Dijkstra, [The Structure of the 'THE'-Multiprogramming System](#), Communications of the ACM, May 1968.

(Additional historical background on [semaphores in Wikipedia](#).)

Q: Dijkstra explicitly states their goals for the THE operating system. How do these goals compare to, say, Microsoft's goals for the Windows operating system? Why do we no longer build operating systems with the same goals as THE?

A: The goal for THE os was to improve the usage and efficiency of processes; but was limited to single users per machine, no shared data base, and only one coding language (ALGOL 60).

Microsoft's goal for the Windows operating system is catered to the users of their product, whether that be students, white collar workers, academia etc.

Dijkstra was building a machine specifically “as a service to the University” with the goal of improved efficiency and reduced turn around time, but Microsoft develops their work with their users in mind, as such, they focus on improving user experience, integrating new technologies, or beefing security.

We no longer build operating systems with the same goal as Dijkstra because we have developed past these goals and have created a society with more breadth of usage for computers, having them be a part of almost every aspect of our lives. The goals of modern operating systems still align with the idea of improved efficiency and performance, but carry a more complex component because computers are no longer used only at Universities for research purposes.

The name of the system described ‘THE’ is a direct reference to the university that Dijkstra was building the os for: ‘[Technische Hogeschool Eindhoven](#)’ in the Netherlands. Our use cases for operating systems have expanded past only academia, so we do not carry the same goals that Dijkstra did all those years ago.

Introduction

Describes Dijkstra's reason for releasing the paper when he did

The Tool and the Goal

a machine's configuration → memory cycle, drum size (data capacity), addressing mechanism, sound system (i/o?), low capacity channels (printers/code input?)

1. Reduce turn-around time for short programs
 - a. Increase efficiency for programs that tend to be stacked up in queues
 - b. batch system
2. Economic use of peripheral devices
 - a. i/o devices like keyboards and storage drives
3. Automatic control of backing store to be combined with economic use of the central processor
 - a. OS controls the drum (hard disk/storage) movement and placement of data
 - b. paging

4. Economic feasibility to use the machine for those applications for which only the flexibility of a general purpose computer is needed, but not the capacity nor the processing power.
 - a. machine can be used for less powerful tasks.
5. Verification of layers - specifically relative to the layers below it

A Progress Report

Says that they planned the project to minimize errors by architecting for all of the possible bugs. Even overconfidently even saying: *“At the time this was written the testing had not yet been completed, but the resulting system is guaranteed to be flawless”* – page 342

However, this also likely led to an early over-engineering of the project, making the planning even more difficult.

A Survey of the System Structure

Storage Allocation:

von Neumann machine $\rightarrow$ CPU + i/o + storage + memory = computer architecture

- *Gier Algol* $\rightarrow$ ALGOL – 60 = old Danish computer from 1962: moved program segments between drum and core while program is running (Dijkstra's description of secondary storage)
- Changed the way memory addresses in physical memory are allocated $\rightarrow$
 - Instead of saving the actual “drum” address he proposed they save the segment + page number, then using core page tables to store the drum address
 - Old method would require whole program to be in memory contiguously, new method allows program to use required number of pages mapped and used, swapping out from core as needed (original index page table)
- Instead of re-writing to the same physical location, the pages are written to the drum by open space and updating the segment table as needed, allowing for much faster writes

Processor Allocation:

- System is organized as a *“society of sequential processes”*:
 - Each user program accepted by the system, each input peripheral, and each peripheral output corresponds to a sequential processes
 - *we have the “segment controller” associated with the drum and the “message interpreter” associated with the console keyboard*
- Making all of the processes run relatively allows them to apply synchronization rules without freezing up the system by delaying one process’s progress.
 - Number of processes running is now independent of the processes themselves (speed, size, execution cost)

System Hierarchy: $\rightarrow$ layers of the sequential processes from above

0. **“scheduler”** determines **timer interrupts** processed & **process states** determined $\rightarrow$ ready, running, & blocked
 - a. abstracted: the CPU, provides processes, & scheduling

1. **“segment controller” (SC)** synced to the drum interrupt (i/o interrupt) and using the segment table in the ram → swaps pages from storage to ram
 - a. abstracted: data segments (modern pages)
2. **“message interpreter” (MI)** allocation of the console keyboard (input) → requires 2 parts (peripherals) to communicate with processes: message interpreter (input keyboard) + console teleprinter (output printer).
 - a. abstracted: console virtualization
3. **peripheral management, buffering input** streams and **unbuffering output** streams → above *MI* to allow processes to return error codes to the operator (programmer)
 - a. abstracted: virtual devices
4. The independent-user **programs** → pass the input to the third level and take the output
5. The **operator** → human

Design Experience

- “Finally we learned the art of reasoning by which we could prove that the *society composed of processes* thus mutually synchronized by each other would indeed in its time behavior satisfy all requirements.”
- They built up each of the levels using machine code on a virgin machine, testing each level and adding portions of the next level up after testing thoroughly.
 - They had to create and test each of the states (ready, running, blocked) and correlate with different interrupts, timer/io/drum/page.
 - Separation of level 0 and level 1 allowed the testers to work without the fear of uncertain timer-interrupt based bugs.
 - And the segment abstraction being separate from the drum allowed them to use RAM correctly even when the drum interface was broken during testing

Conclusion

- Programs can not be tested on all possible states so they had to create a subset of relevant states, but to identify what is relevant they would have to build in regards to the internal mechanism.
- Dijkstra designed a computer architecture of five hierarchical levels to describe what rules would need to be established to *multiprogram*, while believing that more complex projects would drastically benefit from well defined restructuring.

Appendix

- *sequential processes*
 - Used to contrast to electrical signals and circuits being unable to stop
- Synchronization via Semaphores
 - P (wait) – problem (to test)

- V (signal) – verhogen (to increment)
- 2 types of semaphores described
 - Mutual exclusion (binary semaphores)
 - Coordination (between process states)
- Harmonious cooperation
 - deadly embrace – deadlock !

● **What problem does the paper address?**

- The architecture of an OS that allows for multiple processes to run at the time time – “Multiprogramming”
- “The primary goal of the system is to process smoothly a continuous flow of user programs as a service to the University.”

● **How is it different from previous work, if any?**

Since this paper is so old, I think it is different than future works in some notable ways:

- Dijkstra directly talks to the audience in the first perspective, describing his team and how many people are just not intellectually adept enough to understand the system design work that he is covering.
- The verbiage in the sentences is not the same as modern English; but this may be derived from the fact that Dijkstra was Dutch and learned English as a non-primary language.

● **What is the approach used to solve the problem?**

- Choose a project that's complex but can incorporate system updates
- Select a machine and keep the configurations as similar as possible for as long as possible during the experimentation
- Consider that new knowledge != understanding

● **How does the paper support or otherwise justify its arguments and conclusions?**

Dijkstra states two major mistakes they made:

- tried to perfect the installation too early, not planning out “pathology” early enough
 - Made the design too complex early on, making debugging and modifications to the plan much more difficult
- Conceived and programmed the major part of the system without considering the thought of debugging it.
 - They planned the architecture **to minimize the need for debugging** because of his fear of timer-interrupts caused by hardware leading to unreproducible bugs caused by the interrupts.
- Created testing methods by building up one level at a time, testing the level and then adding portions of the level above, testing each upgrade as they go.

● **Was the paper, in your judgement, successful in addressing the problem?**

- Yes, the paper successfully describes the structure of a successful multiprogramming operating system
- The methods described in this 57 year old paper are still taught to this day, and though they have since been upgraded, they still ground the techniques we have developed
- For example: CPU scheduling is built on level 0 and paged virtual memory is level 1, both of which are fundamentals of all operating systems
- From the looks of the paper, Dijkstra and his team were successful in creating a system that improves machine processing speed and efficiency.

P. B. Hansen, *The Nucleus of a Multiprogramming System*, Communications of the ACM, April 1970.

Q: How does synchronization in the RC 4000 system compare with synchronization in the THE system?

RC 4000 system uses a message passing buffer primitives while THE system uses locking mechanisms for shared memory powered by semaphores.

The RC 4000 kernel is designed with primitives for interprocess communication built up on a message passing system controlled by the parent process sending data to their child process, and a buffer queue to ensure that the order of messages is passed properly. The parent process controls the creation, ready, stop, and delete of their children processes, requesting the scheduler to start the child process as needed.

xSynchronization for parallel processing is applied when multiple processes are attempting to communicate, leveraging the send message, wait message, send answer, and wait answer primitives. This ensures that memory does not need to be shared between processes, but parent processes can allocate and unallocate for the children processes.

THE system relies on Dijkstra's definition of semaphores; processes are implemented with an initialized mutex value (of 1 or 0) that determines if the process has permission to use shared memory. This way only one *running* process can access mutual (shared) memory.

Introduction

- Describes how multiprogramming in computers is limited by modes of operation, such as “*batch processing, priority scheduling, real-time scheduling or conversational access*”
- They aim to replace single-processing operating systems with a nucleus (**kernel?**) that can be extended to new operating systems.
- Overall paper is focused on discussion (policy) not implementation (mechanism)

System Nucleus

1. Assign a specific meaning to what a ‘process’ is.
2. Select primitives of sync and transform of information between parallel processing
3. Rules of dynamic creation, control, and removal of processes.

Process

program - collection of instructions describing a computational process

peripheral device - hardware device connected by a data channel and device number

document - collection of data stored on physical medium

external process - the input/output of a given document by unique *process* name

internal process - execution of the instructions described by a *program*

Process Communication

Dijkstra introduced semaphores for multi-process communication, but can't stop processes who break the rules. They propose a message buffer & queue system.

- send message (receiver, message, buffer) wait message (sender, message, buffer)
- send answer (result, answer, buffer) wait answer (result, answer, buffer)

External Process

- External processes are created at the request of internal processes when attempting to deal with input and output from a peripheral device.
- *reservation* and *release* primitives are established to ensure singular access to documents.

Internal Process

- Describes the creation, control, and removal of internal processes (`fork()` ;)
- Parent process can load a program into the child process and *start* it, causing it to share its computing time with other active processes under the parent process
- Parent can also remove a child, reassigning its storage to other processes

Process Hierarchy

- OS starts with one process *S* which is the parent process of all other processes, in a tree like structure,
- each process can allocate children portions of their their resources (storage and message buffers)
- These processes can start, stop, and remove its own children, returning resources after removal

Future planning for operating systems

1. New OS can be implemented without modification of the nucleus
2. OS can be replaced dynamically, meaning multiple can be active simultaneously
3. Standard programs and user programs can be used in different OS, with a common agreement of communication between parent and child.

Implementation

RC 4000 is a 24-bit, binary computer with typical instruction execution times of 4 μ s.

- *“Multiprogramming is facilitated by program interruption, storage protection, and privileged instructions.”*
- Primitives are operated in the nucleus within an uninterruptible mode – with established limits to system responses for real-time events (IPC), 0.6 msec for sending message and answers, and 0.4 msec for wait answer and message.
- States that the even distribution of time usage for the primitives implies that only hardware improvements could lead to significant efficiency increases.
- create 3 msec, start 26 msec, stop 4 msec, remove 30 msec

Conclusion

- “*process concept, the communication scheme, and the dynamic creation and structuring of processes.*”

● What problem does the paper address?

- The architecture of a kernel that facilitates communication between processes on a multiprogramming system, specifically describing what a process is, and how processes inter-communicate

● How is it different from previous work, if any?

- Previous work, like Dijkstra's work from 1968 describes how semaphores can be used to allow for multi-programming. Hansen states that semaphores are the first step to establishing mutual exclusion, but more specific regulations are required for inter-process communication.
- Clarifying what a parent process is, stemming from the original process created by the system on startup, and connecting the branches of children created with rules for who is able to access specific data relative to the parent.

● What is the approach used to solve the problem?

- Hansen's approach is to establish the policy and clearly define the primitives involved in the system.

● How does the paper support or otherwise justify its arguments and conclusions?

- The paper specific direct values for the words for each portion of the system benign used, stating their proportions of the access speed as well as specific msec costs for the primitives described.

● Was the paper, in your judgement, successful in addressing the problem?

- The paper successfully demonstrated the creation of a kernel for an OS, with some possible primitives required for a multi-programming system.
- They define what a process is, specifically splitting up *internal* from *external* processes, what *peripheral devices* are and their relationship to data streams, connecting that to *documents* that are used by the internal processes.
- Most importantly, they clearly explain what the rules are for communication between internal processes using message queues and buffer queues.

Lecture Content

Structure of Modern OSes

- Linux → 2 layer system
 - User-Level
 - Kernel
- Windows → 2 layers with sublayers:
 - User Apps
 - Win32 Subsystem
 - Kernel
 - Hardware Abstraction Layer

Hardware Trends

- Absolute number
- Change over time (trend)
- Relative change – how does the trend compare to the other components?

component	1968	2025	factor change
clock speed	400 KHz	3 GHz	7,500x
memory capacity	32 kb	32 GB	1,000,000x
storage capacity	512k words	10 TB	20,000,000x
storage revolution	40 ms	7,500 rpm 8ms	5x
paper readers	1000 char/s 8 kbits/s	300 Mbps	37,500x

THE Summary:

- Layered OS structure
- abstraction : sequential processes
- Semaphores for synchronization
- Correctness with proofs and testing

Nucleus Paper

Goals:

- Support multiple OSes

Components of Nucleus

- Processes & process control – create, start, stop, remove
- IPC via messages
- Interrupts and exceptions
- Scheduling

OSes on top of the Nucleus

- memory management
- processes management - managing parents and their children

Abstractions in Nucleus

- processes:
 - internal – our understanding of processes: stack, code, heap
 - external – wrapper around a hardware device
- messages:
 - How processes can communicate with each other

Principles of OS design: **economy of abstraction**

- Useful to design small amount of abstractions that are heavily reused:
 - Processes and messages to allow for apps and inter-app-communication
- External processes ; uses process abstractions for peripheral devices
- Synchronization via messages
 - messages for sync is to debug easier
- clock process

Nucleus Summary

- Goal: flexible OS design
- Structure: small nucleus, OSes layered on top
- Synchronization via message passing
- Small number of abstractions

History - 10/2

Q: What features in TENEX are reminiscent of features in Unix?

- ## 8.1 Influences

The *fork* operation, essentially as we implemented it, was present in the Berkeley time-sharing system [8]. On a number of points we were influenced by Multics, which suggested the particular form of the I/O system calls [9] and both the name of the Shell and its general functions. The notion that the Shell should create a process for each command was also suggested to us by the early design of Multics, although in that system it was later dropped for efficiency reasons. A similar scheme is used by TENEX [10].

- Hardware was more extensible than today (modifiable at hardware level)

- different terms for the same thing
 - Monitor → Kernel
- Same terms for multiple things:
 - Virtual machine:
 - Hardware (VMWare)
 - Language (Java – JVM) - bytecode
 - OS syscall interface (TENEX)

- virtual machine
 - Virtual memory, multiple processes, file system
- modularity, maintainability, ease of implementation
- Good human engineering
 - Executive
- Sharing and isolation
- Backwards compatibility
- Efficiency

- Address spaces
 - Separate process and OS spaces
- Virtual memory maps
 - multilevel page table – index table specifically
- BBN pager
 - Component of a MMU – similar to the TLB
- Virtual Address Translation
 - virtual address → page table → physical address
- TLB virtual address → TLB → yes → physical address
 no: ↓
 page table → physical address

BBN Pager

- 9 bits (virtual address) → 11 bits (physical address)
- weird for modern standards, one process can't even use all of the address space

Sharing

- Pages of memory
- Using page table entries
 - Direct – default case : one page virtual to one page physical
 - Indirect – multiple processes can use the same physical page
 - Secondary pointers point to the direct table entry for the physical page
 - Shared – page cache or buffer cache
- Copy-on-Write
 - Unwritten: $[\text{proc1}] \rightarrow [p] \leftarrow [\text{proc 2}]$
 - 2 writes: $[\text{proc1}] \rightarrow [p] \quad [\text{proc 2}] \rightarrow [p2]$

Backwards Compatibility

goal: binary compatibility

- package/library – JSYS: allows additions of system calls
- Package runs on user level, allowing for usage of old-syscalls from DEC10/50
 - JSYS was a wrapper for old → new syscalls

Exec (executive) = shell

- Command completion (used the escape key)
- Abbreviations

File System

- hierarchical system, did not allow nested directories
- $C:\text{dir}\backslash\text{file.ext.version} = \text{device} \backslash \text{directory} \backslash \text{name} . \text{extension} . \text{version}$
- Versioning was important question, do we save to file or user level apps

Scheduling

- Priorities, round-robin

Summary

- Virtual memory – sharing, copy-on-write
- Time-sharing system
- Support for backward compatibility

D. M. Ritchie and K. Thompson, *The UNIX Time-Sharing System*, Communications of the ACM, July 1974.

Q: What aspects of Unix as described in the 1974 paper do not survive today, or have been considerably changed?

Modern versions of Unix (or even Linux) are very different from the one described by Ritchie and Thompson in two noticeable aspects: size and security.

The authors mention repeatedly that they were building their operating system around the PDP-11 system with a constraint on size, allowing them to run it on systems that cost as “little as \$40,000”. The pricing of computers between the modern age and that era is insane, considering \$40,000 as a little amount of money would be unheard of today because of our easy access to technology.

Additionally, the Shell implemented by Ritchie and Thompson would initiate an instance of the operating system with a simple username and password, where the password file would be able to be manipulated to launch files directly, circumventing any protections applied specifically to the initial login page.

Key Takeaways

- Describes the modular file system implemented into UNIX
 - 3 types of file abstractions: ordinary, directories, and special files
 - Ordinary = normal files (text, code, etc)
 - Directories = pointers to other files (implemented with inodes)
 - Special = drives that were physically mounted, using the files abstraction
- Defined a multi-user system focused on ease of use, especially with the Shell program that allowed easy development, execution, and testing of programs
- Described important commands that defined the course of OS research:
 - open, create
 - read, write
 - seek, link, pipe
 - execute, fork
 - wait, exit
 - sh, ls,
 - <, >, |, &

UNIX

Influences

- TENEX
- Berkley's time-sharing system
- Multics – worked on by Ritchie and Thompson, who tried to fix the dumpster fire it was

Goals:

- Easy to use – especially for developers (interactive)
- Cost effective & inexpensive to allow for more users to access
- Easy to maintain – using it as their own platform for development
- Unifying abstractions (simple set of reused abstractions)
 - Elegance of design
 - Everything is a file

File System

File data model

- Before :
 - Specific structure imposed on files & required pre-declared file sizes
- Unix :
 - Normal files - uninterpreted collection of bytes & variable file lengths
 - Directories - specific type of file, with special rules

File System APIs

- Before :
 - No buffer cache in the Kernel
 - Separate APIs for sequential vs random files
 - Asynchronous APIs, required waiting for files to get back to user
- Unix :
 - Unified the API style : `write(fd, buffer, count)`
 - seek for random access files
 - buffer cache in the Kernel – for reads and writes

File names and directories

- Before :
 - One directory per user
 - Directory represented differently on disk
- Unix :
 - Directories stored as files, reusing the file abstraction
 - File open flags to determine if the files are being used by processes
 - Hierarchical name spaces
 - `/root/dir1/dir2/dir3...`
 - Mount external drives into existing system

Devices

- Before :
 - per-device kernel calls
- Unix :
 - device-independent I/O
 - Everything is a file again - specifically “special files”
 - `ioctl` - device specific functionality

Protection model

- Before :
 - List of users (Multics)
- Unix :
 - permission bits stored with the file name for read, write, and execute
 - 6-bit scheme owner/other R/W/X
- Today :
 - 9-bit scheme owner/group/other R/W/X

Processes

- Before :
 - One process per user
- Unix :
 - Many processes per user
 - `fork(label) ;` – label for where the child should jump to
 - `exec() ;`

Shell

- Before :
 - Built into the kernel, not customizable
- Unix :
 - Shell is user-level program, allows much more flexibility
 - I/O redirection
 - `stdin` and `stdout` cover `fd 0` and `fd 1`
 - `$ command < in.txt > out.txt`
 - Example execution loop for shell

```
while (1) {
    read command
    fork();
    if (child) {
        -reassign stdio & stdout
        exec(command)
    }
    wait(child)
}
```

APIs for Process Creation

- Unix :
 - `fork()` and `exec()`
 - pros of `fork()`:
 - Easy to use, simple to execute
 - Easy to share parent & child resources
 - Copy-on-write reduces overhead for files

- cons of fork():
 - Can be inefficient/slow
 - Complex to implement in kernel (lots of data/attributes to copy)
 - Not thread safe!!
 - Insecure because of copying
- Windows :
 - CreateProcess(name, cmd, attribs, ...)

Summary

- Unifying abstractions → everything is a file
- Careful decomposition of build up

Protection - 10/6

W. Wulf, E. Cohen, W. Corwin, A. Jones, R. Levin, C. Pierson, and F. Pollack, [HYDRA: The Kernel of a Multiprocessor Operating System](#), Communications of the ACM, June 1974.

Q: How is a Hydra procedure different from the procedures we are familiar with in a typical language and runtime environment?

[answer](#)

Zac Blanco Notes:

HYDRA

- Use C.mmp multiprocessor, 16 cores (PDP-11's), 32 million addressable bytes

Specific considerations for Hydra

1. Multiprocessor environment
 - not many implementation of multiprocessor systems - should be conservative
2. Separation of mechanism and policy
3. Integration of design with implementation methodology
4. Rejection of strict hierarchical layering
5. Protection
6. Reliability

3 critical concepts

- Procedure
- LNS (Local namespace)
- Process
- **capability** consists of a reference to an object together with a collection of “access rights” to that object.
- Make a procedure call (system call?) which creates a new LNS from the callee based on capability.
- *Process* is the smallest entity schedulable for execution
- argue hierarchical structures are bad for security
- each object has a capability which is only modifiable by the kernel and data, representing the data behind the object

kernel provides the *walk* primitive

- input: capability and nonnegative integer
- output: the capability that occupies the cap in the part of the object named by the parameter capability

Users have access to procedures, which also have access to some amount of object data

Notes from Lecture

- Hydra is based on security and capabilities
- separating mechanism (the ability to do an action) from policy (condition enforcing a mechanism). -

Most systems today use ACLs, hydra proposes capabilities

- ACL

- example: Files today have ACLs which are owner, group, other, with policies RWX. When showing up to perform a mechanism, need to show identity in order to perform an action
 - Data stored on the *object* with the list of “people” who can access and what they can do.
- Capabilities
 - Issued a key which “unlocks” ability to perform action on a number of different objects
 - Data associated with available actions is a list of objects with the specific actions allowed for that user.

Local Namespace

- Represent an execution domain
- contains list of pointers (capabilities) that can be accessed by that namespace

Procedures

- code, operations, and objects
- templates: define type and capability

B. Lampson, [Protection](#), Operating Systems Review, January 1974.

Q: What are the concepts in Hydra that correspond to Lampson's definitions of "Domain", "Object", and "Access Matrix"?

answer

Zac Blanco Notes:

- Security mechanisms are far and wide
 - user/kernel mode
 - memory bounds checking registers
 - filesystem ACLs
- Security should be abstract. Many previous mechanisms focused on user separation. More now on program separation. Safety mechanisms enable feasibility of large systems as well as commercial applications.

Message passing system analogy

- processes separated by hardware (inefficient)
- message from A to B is ok
 - what if B is malicious or unreliable? Wait for a message from C which *is* reliable.
 - Q: but how can we verify C is reliable if B isn't?
- Each process gets the identity of the caller, so it can accept or reject.
- Verifying identity is difficult
- Object system
 - Set of domains
 - access matrix

Contains objects

- definition of object up to reqs of a system (i.e. processes, domains, files, segments)
- Access of domains to objects is determined by an access matrix
- each element of the matrix specifies the a set of strings, "access attributes", (e.g. rwx) which label the access abilities of a domain for a given object

Can implement capability lists with a matrix of

$A[d, A[d, x]]$

- Generalized idea of users v files access with ACL or capability list
- users or files can be "domains", a set of access abilities exist under each combination of domains.

Protection - 10/9

J. H. Saltzer, [Protection and the Control of Information Sharing in Multics](#), Communications of the ACM, July 1974.

Q: Compare and contrast protected subsystems in Multics with procedures in Hydra.

Multics

Goal

- uniform protection model – applies to everything in the system
 - Derives from static file system
 - When running, specific methods enforce protection

Principles

- Check every access – be careful how you cache information
- Permission rather than exclusion – (whitelist not blacklist)
- Design is not secret – don't assume attackers won't understand architecture
- Principle of least privilege – easier to ask for more privilege than have too much
- Usability – human interface should be easy to use

Protection

- - how can protection be efficient
- - login – establish UserID (principle identifier)

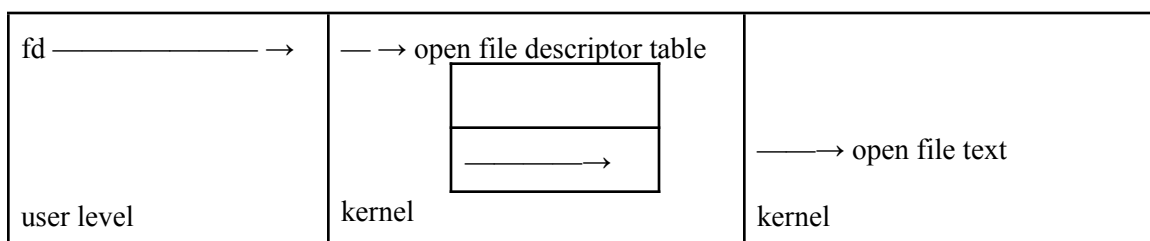
File System

- Protection is derived from the file system – holds the ACL
- Access Control Lists (ACL) → static
 - searching through these lists – especially on every file access – gets expensive
 - Capabilities → dynamic
 - Permission checks done at runtime – derived from ACL

File Example:

possibleAPI: read (“file.txt”, buffer, length, ...)
 find acl, scan acl, check perms on file.txt → then pass read

modernAPI: fd ← open(“file.txt”,permissions ...)
 read(fd, buffer, length, ...)
 file descriptor (fd) serves as capability



Segmented Memory Model

- Split virtual memory into multiple segments:
 - Process: code, stack, files, heap, libraries, ...
 - OS: FileSystem, ...
- Descriptor table holds correspondents for each segment,
 - each segment has their own permissions connected to the ACL
 - Virtual addresses represented as 2 components:
 - Segment number
 - Offset
 - Very similar to paging, but segment size is not standardized
- segment descriptors → basically a capability

physical address	RWX	call?	ring?
pointer	rights		

Protected Subsystems

- File System – create, read, write
- gates – entry point into a subsystem – similar to a generalized syscall
- ring – 8 level generalization of levels → ex. user-level vs kernel-level
 - Bottom level has the most privilege; top has least privilege
 - ex: coreOS, drivers, file system, libraries, applications, user ...

Modern vs Old

Hydra

- Reject hierarchy
- Also uses protected subsystems and capabilities

Modern

- Similar to multics rings – but much fewer rings
- Syscalls are similar to gates

Summary

- Protection in file system, based in access control lists
- Protect memory accesses using descriptors (capabilities)

G. C. Hunt and J. R. Larus, [Singularity: Rethinking the Software Stack](#), ACM SIGOPS Operating Systems Review, April 2007.

Q: How does the language-based approach that Singularity takes compare and contrast with other approaches to protection we have discussed so far?

Singularity (2007)

- authors : Operating Systems + Programing Languages/Compilers perspective

Motivation/Goals

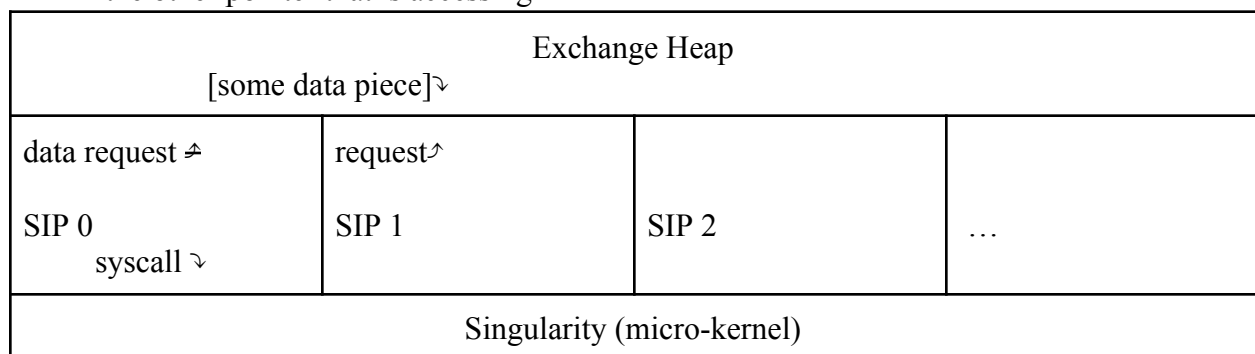
- Avoid bottlenecks caused by physical checks
- Dependability, and reduction of runtime errors
- Security vulnerabilities – especially from additional libraries and drivers
- Failures

Approach

- Leverage language features in the operating systems
 - ex: using Java to stop buffer overflows (built in bounds checking)
- Verification – in the sense of THE verifying their code

Design Components

- Software Isolated Processes – SIPs
- Contract-Based Channels – CBCs
- Manifest Based Programs – MBPs
- one address space – no user-level or kernel-level
- [domain] $\rightleftharpoons$ message $\rightleftharpoons$ [domain] [SIP] $\rightleftharpoons$ CBCs $\rightleftharpoons$ [SIP]
- data in SIP is private, can not interaccess information without request
- only 1 pointer is allowed to access the data, when new one pointer takes over, invalidates the other pointer that is accessing



- Type systems
- Software verification & runtime checks
- Similar to *smart pointers* in C++ and *ownership* in RUST

Drawbacks

- Shared address space for all processes
- everything has to be in a specific language

Summary

- Software based isolation
 - programming languages
 - verification

- What **problem** does the paper address?

- Security vulnerabilities from unsafe interactions between applications and drivers
- Application failures caused by lack of isolation and robust communication protocols
- Inability to verify system-wide properties before execution due to dynamic, loosely coupled designs

- How is it **different** from previous work, if any?

Unix: Uses hardware for process isolation and is implemented in C. Singularity uses software isolation (type-safe language) and statically verified message contracts.

TENEX: Relies on hardware memory protection and unsafe languages. Singularity uses software-enforced isolation and static manifests for safe deployment.

Hydra: Implements object-capabilities and hardware isolation but uses unsafe languages. Singularity ensures object isolation through language safety and static verification.

THE: Introduced layered design, but had no language safety or automated verification. Singularity uses modern type safety and static analysis for boundaries.

- What is the **approach** used to solve the problem?

Software-Isolated Processes (SIPs): Processes are isolated using type and memory safety guarantees from the language and runtime, not just hardware.

Contract-Based Channels: Communication between processes is done with channels whose protocols are statically defined and verified at compile time.

Manifest-Based Programs: Applications declare their requirements and behaviors up front in manifests, enabling static checking of system-wide properties before execution.

- How does the paper **support** or otherwise justify its arguments and conclusions?

1. Working OS Prototype with SIPs, Channels, and Manifests

- Software-Isolated Processes
- Contract-Based Channels
- Manifest-Based Programs.

2. Dependability Improvements (Crashes and Vulnerabilities Reduced)

- SIPs provide strong boundaries, no shared writable memory, sealed code spaces, and enforce failures to local processes.
- SIPs are “cheap enough” to provide robust, fault-stop boundaries.

3. Performance Overheads Demonstrated as Acceptable

- **Section 2.1, Table 1:** Compares process creation, message passing, API call, and context switch costs between Singularity and other OSes.
- **Section 4.2.1** (Quantifying the Unsafe Code Tax): Describes experiments (Figure 5) showing the overhead of hardware versus software isolation and safe code.

4. Case Studies and Experiments with Static Verification

- **Section 2.2, Paragraphs 6–8:** Describes how channel contracts and contract verifiers caught real bugs in legacy communication code.
- **Section 2.3, Paragraphs 2–5:** Shows how manifests enable properties like safe driver installation and enforcement of behavior before execution.

5. Comparisons to Traditional OSes

- **Section 1, Paragraph 2:** Mentions that traditional OSes rely on legacy architectures and languages.
- **Section 3:** Discuss the stripped-down ABI and comparison with UNIX/Windows in terms of complexity and entry points.
- **Section 4.2:** Discusses hardware isolation mechanisms (like those in traditional OSes) and their costs.

- Was the paper, in your judgement, **successful**, in addressing the problem?

Synchronization - 10/14

C. A. R. Hoare, [Monitors: An Operating System Structuring Concept](#), Communications of the ACM, October 1974.

Q: What are "monitor invariant" I and "condition" B, and why are they important in the discussion of monitors?

Monitor Introduction

- Monitor {
 - private data;
 - A() { ← acquire lock
... wait(); ...
} ← release lock
 - B() {
... signal(); ...}
}
- only one thread can execute in the monitor at once
 - ensured by using a built in lock inside the monitor
- compiler inserts the lock calls

2 main uses for synchronization

- mutual exclusion
- coordination (between threads)

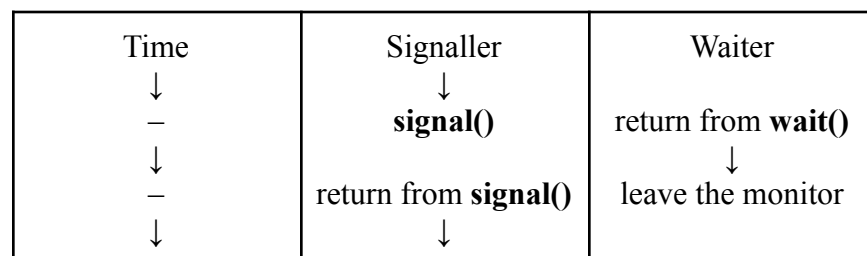
Condition Variables

- Goal: allow threads inside a monitor to wait
- **Wait ()** – suspends the thread, releases the lock
- **Signal ()** – wake up a waiting thread; thread reacquires the lock

Hoare vs. Mesa Semantics

Hoare Semantics

- Wait – puts the thread on queue,
- Signal – immediately calls the waiting thread to start running



invariant I – consistency of the data structures that are managed by the monitor

- When can a program leave the monitor?
 - exit, wait, signal
- When can you assume the invariant is true?

- When you can enter or resume the monitor.

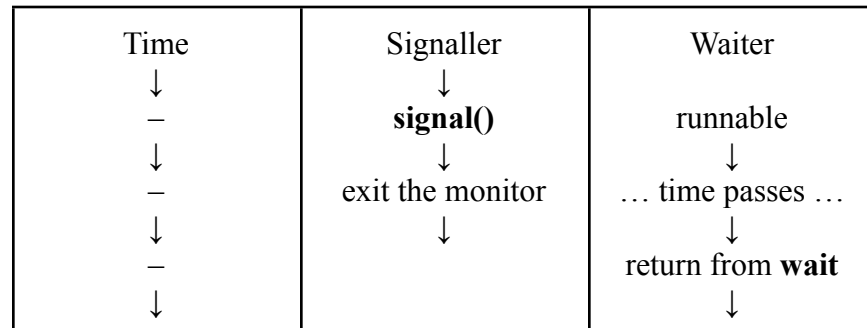
condition B – situation that the waiter is waiting for (condition to wake up)

- When can we assume the condition is true?
 - Thread returns from **wait()**;
 - Need to ensure its true before calling signal

$B \mid I \text{ wait}() \mid I \mid B \rightarrow I \text{ true before and after calling wait}$
 $\downarrow \downarrow \rightarrow B \text{ becomes true some time after wait, but before signal}$
 $I \mid B \text{ signal}() \mid I \rightarrow \text{both true before because directly from wait releases}$

Mesa Semantics – weakened the semantics but made condition use cases more broad

- Wait – puts the thread on the queue
- Signal (notify) – puts the thread on the runnable queue, continues



- On return from wait, condition might not be true

Hoare Semantics	Mesa Semantics
<pre>if (!condition) { wait() }</pre>	<pre>while (!condition) { wait() }</pre>

$I \text{ wait}() \mid I \rightarrow I \text{ true before \& after wait, unknown status for B}$
 $\downarrow \downarrow \rightarrow B \text{ no need to be true bc its repeatedly checked by while}$
 $\neg B \text{ signal}() \mid \neg B \rightarrow \text{same state for I as before signal; can't assume for after}$

B. W. Lampson and D. D. Redell, [Experience with Processes and Monitors in Mesa](#), Communications of the ACM, February 1980.

Q: Compare and contrast synchronization in Java with Hoare monitors and Mesa monitors.

Compare and Contrast: Hoare vs Mesa vs Java

	Hoare Monitor	Mesa Monitor	Java
monitor lock	All methods	entry methods, internal/external methods,	synchronized key word
condition variables	explicit	explicit	implicit singular cv explicit cv (newer)
wait();	Hoare semantics	Mesa	Mesa (general pl)
signal(); abstraction	Hoare signal();	Mesa notify(); / broadcast();	Mesa notify(); / notifyAll();
granularity (scope)	coarse (whole os, file system, etc)	monitor class + record	multiple (based on synchronized classes)
abort	did not address	unwind handler, throws exception and traces	programmer defines exception handlers
nesting	did not address	Let first process hold the lock	need to manage deadlocks
	A holding → wait() lock ————→ signal() B 	deadlocks very possible, if A holding lock, and B is called to wait(), but holds the code to signal in A, B would get deadlocked out waiting for A to start but it would never be able to be its locked out	
scheduling	Immediately executes code upon signal	Adds process to queue but does not start it	Does not guarantee immediate scheduling
ordering	Resumes specific waiter in order of cond.queue	Resumes some waiting process at convenient future time	Indeterminate; awakened thread returns to competition for lock

Trends over time

- Hoare and Mesa – rely heavily on compiler support
- Java – compiler support for implicit CV with `synchronized` classes
 - programmer managed locks with explicit CV
- C++, Rust, Python – don't have monitors in the conventional sense
- has condition variables and locks,
- but managed by programmer
 - (acquire and release locks)

Conclusion

- Language support for synchronization with monitors
- Hoare vs. Mesa semantics
- Practical challenges (from Mesa paper) – aborts, nesting, etc.

Virtual Memory - 10/16

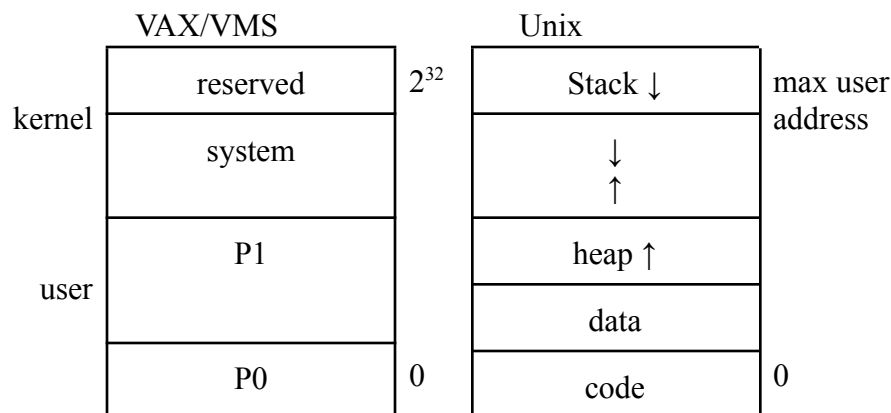
H. M. Levy and P. Lipman, [Virtual Memory Management in VAX/VMS](#), IEEE Computer, March 1982.

Q: The paper states, "VAX/VMS, then, is a collection of procedures that exist in the address space of each process." Explain in your own words what this statement means.

VAX/VMS

Goal:

- Support different hardware (PDP11 was popular, others were rising)
- Handle resource constraints
 - Limited memory
 - Slow I/O
 - Support 32-bit address spaces (4gb)
 - 250kb – 8mb physical memory



Paging

- Fixed-size pages, 512 bytes each
- Compared to variable-sized pages in Multics
- Modern systems use multi-leveled paging tables instead of linear arrays

31	30	29 – virtual page number	08 – bytes offset
----	----	--------------------------	-------------------

31/30 = region bits; 00 = P0 region, 01 = P1 region, 10 = System Space, 11 = Unused

Page Tables

- Linear arrays of Page Table Entries (PTEs)
- Not multi-level (yet)
- Hardware support –
 - base register – virtual address (P0, P1, physical address (system))
 - length register
- segmentation fault (from Multics) if attempting to access illegal memory
- context switch: update registers

Space Efficiency

- How to minimize space used for PTEs? (not useful for users)
- Swap process page tables
- Page tables for the operating system?
 - Keep it in physical memory
- Translation Lookaside Buffer (TLB)
 - Hardware support for caching pages
 - Improved since TENEX
 - Split into user and kernel entries
 - to ensure kernel pages couldn't be removed bc of high user-memory usage

Efficient Paging

- Local page replacement
 - Replace a page from the process requesting a new page
 - FIFO policy – low overhead and easy to implement
- Resident set – pages allocated per processes
- **Page Caching**
 - Free list– don't need to be written to disk
 - Modified list – need to be written to disk
 - Fault on either list brings the page back into resident set
 - Second chance algorithm
- **Clustering**
 - slow I/O, small pages → read/write are slow!
 - read – batch at a time
 - write – collect many to write at once

Summary

- process address space – inherited by UNIX
- memory management and paging efficiency improvement techniques

R. Rashid, A. Tevanian, M. Young, D. Golub, R. Baronn, D. Black, W. Bolosky, and J. Chew, [Machine-Independent Virtual Memory Management for Paged Uniprocessor and Multiprocessor Architectures](#), In *Proceedings of the Second International Conference on Architectural Support for Programming Languages and Operating Systems*, October 1987.

Q: Why does Mach support copy-on-write, and how does it implement it?

Test

Mach

- Tevanian → Apple; became CTO
 - Mach VM → MacOS VM
- Rashid → founded Microsoft Research (MSR)
- IBM **RT PC**
 - **RISC** – Reduced Instruction Set
 - **PC** – Personal Computer
 - POWER, RS/6000, PowerPC, better for embedded systems
 - PowerPC – used for Xbox360, PS3

Goal

- Virtual memory that's machine independent
- Hardware differences:
 - Single vs Multi-processor
 - non-uniform memory
 - segmentation vs paging
 - TLB / memory management
- good/improved performance
- better access for sparse address spaces
 - Memory-mapped files
 - Multiple threads
 - Shared libraries
 - Future security benefit of randomized address space layout allocation

Address map

- Stores a doubly linked list of address map entries
- 0 : code ↔ data ↔ heap ↔ file ↔ stack : 2^{32}
- Each region stores protection, range of addresses

Resident Page Table – inverted page table

- One entry per physical page
- Tracks all physical Mach pages (wrapper for physical pages)
- Useful for page replacement

Memory objects

- Backing store for a range of virtual addresses

- keeps track of where a region of memory is stored when it's on the physical device, rather than when it is in virtual memory
- customized pagers
 - Linux: userfaultfd

pmap

- Hardware-dependent data structure, needs to be custom designed for the system
- Hardware differences:
 - Structure of the TLB
 - How TLB misses are handled
 - paging vs segmentation
- “soft state” – is maintained in other data structures, but pmap makes it efficient

Summary

- machine-independent representation of the system.

Distributed - 10/21

J. K. Ousterhout, A. R. Cerenson, F. Douglass, M. N. Nelson, and B. B. Welch, [The Sprite Network Operating System](#), IEEE Computer, February 1988.

Q: What are the technology trends that motivated Sprite's design, and how do you think these trends have evolved since then?

Goals:

- **larger physical memory**
 - Caching benefits
- **networking**
 - workstations connected by a network
 - sharing (data, processes, environments etc.)
 - Finding files
 - Single global namespace (prefix tables)
 - More difficult on multiple machines
 - using idle machines
 - Process migration (RPC)
 - admin/management of cluster configurations
 - Transparency
 - Want users to think everything is single machine (files/programs)
- **multiprocessor workstations**
 - They expected multiprocessors to rise within a few years, took like 10 lol
 - Parallelism and sharing
 - Shared fork
 - OS → multiprocessors
 - not as easy as locking/monitor on one core
 - subsystems + *monitors* for locking (RCU)

Caching

- Caches file data on both the client and the server
 - Not the first system to do this, but Sprite is very well known for caching
- Client A []
- inconsistent ↓ File Server [] [] - file blocks (4kb)
- Client B []
- Concurrent sharing - Sequential sharing
 - Disable caching - Version numbers (with time stamps)
 - backing store → regular files

Physical Memory Allocation

- File buffer cache vs virtual memory
 - Dynamically updating size for memory vs buffer, based on balancing oldest page on both sides

File buffer cache	Virtual memory
-------------------	----------------

Process Migration

- Allow processes to migrate across workstations
- goal: make this transparent to the users, processes should move without affecting users
- `pmake` – parallel make, allowed processes creation across machines
- challenge: kernel calls – how to do without telling the user it's happening elsewhere?
 - Forward the call back to the home machine
 - modern abstraction is to use VMs and move them across machines

Shared Fork

- Processes in a shared address space = **threads**
- Unix: share code
- Sprite: `shared_fork` → share code and heap data
 - Don't share stack bc then the execution is the exact same (not parallelizing)
- Modern threads: `pthread_create` → share code and data

Summary:

- file caching protocol
- identify technology trends for system design

Y. Shan, Y. Huang, Y. Chen, Y. Zhang, [LegoOS: A Disseminated, Distributed OS for Hardware Resource Disaggregation](#), In *Proceedings of the 13th USENIX Symposium on Operating Systems Design and Implementation (OSDI'18)*, October 2018.

Q: Which resources in LegoOS are partially disaggregated vs. fully disaggregated? Why?

Monolithic Server

- [CPU] [CPU] [CPU] [CPU] → [BUS] ← [DRAM] [DRAM]
[Disk] ^
[GPU] ← [PCIE BUS] - - - - ^ → [NIC] ————— → datacenter network
- cache coherent interconnects
- high performance interconnects

Hardware Disaggregation

- Separate all of the components so they are not interdependent
- [CPU] [CPU] ————— → |
[DRAM] [DRAM] ——— → |
[Disk] ————— → |
[GPU] ————— → [NIC] ————— → datacenter network
- organized hardware into “blades” of each type

Benefits

- Resource elasticity – scale resources independently
- Specialized hardware, for example new computing models
- Better reliability – Fine grained failure domains (fix blades instead of machines)
- Improved resource utilization –
 - Bin-packing problem → can’t optimally use specific needs bc of lack of bins
- **performance is non-goal**, hopefully other benefits cancel out the slight loss

Challenge:

- interconnect : 500Gbps, 50ns latency
- datacenter network : 40Gbps. 5μs latency
- today: 100Gbps, few μs latency

LegoOS

- SplitKernel – uses monitors per component
- pComp, mComp, sComp

pComp	mComp	sComp
CPUs [] [ExCache 4gb] NIC —————	Memory [DRAM] —————NIC—————	Storage [disks] —————NIC
virtual addresses	page tables/ TLBs vAddr → pAddr file buffer cache	

- Removing full memory from CPU created inefficiency but adding small ExCache in the CPU component improved the performance greatly
- Page tables and TLBs are stored on **mComp** so that **pComp** can focus on virtual addresses until **mComp** has to deal with physical memory
- 2 level memory management
 - vRegion
 - vma trees
- Implementation – commodity hardware (used on monolithic servers with disabled parts)
- Evaluation
 - Small working sets – performance was a little bit worse (within 2x)
 - Large working sets – performance was better
 - data too much for one machines memory, faster to rpc than get from disk

Adoption

- Disaggregated GPUs
- Disaggregated storage - AWS S3, DynamoDB
- Disaggregated memory - active research area - CXL
- Machines are larger today – makes bin packing problem easier

Summary

- Split Kernel approach – pComp, mComp, sComp
- Explores extreme point in a design space → pushing ideas to a new end

Extending - 10/23

H. Haertig, M. Hohmuth, J. Liedtke, S. Schoenberg, J. Wolter, [The Performance of Micro-Kernel-Based Systems](#), In *Proceedings of the 16th Symposium on Operating Systems Principles (SOSP'97)*, October 1997.

Q: Compare and contrast the L4 microkernel with Nucleus and Hydra in terms of their goals to provide a basis on which higher-level OS functionality can be implemented.

How can we take abstractions from the operating system and bring them up to user-level?

Extending the OS

Goal: Move OS functionality to the user-level.

Monolithic OS	Mach 90s	Mach v2, L4
[App] ← user-level — — — — trap — — — — [OS] ← kernel-level	servers: [vm] [fs] ... [App] — — — — — — — — — — [μKernel]	[OS] [App] [App] — — — — — — — — — — [μKernel]

- benefits of μ Kernels
 - easy to update
 - easy to debug
 - run many diff OSes
- drawbacks:
 - messaging overhead

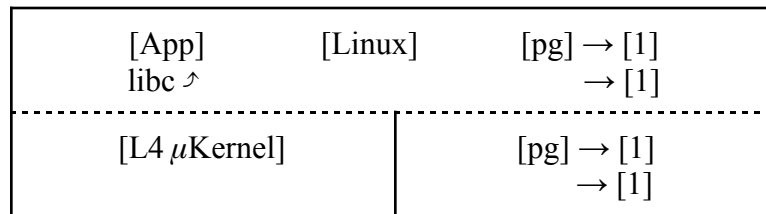
L4 - Goals:

- Design a μ Kernels with good performance
 - Only μ Kernel at the time was Mach – not that efficient
- Run a standard OS on a μ Kernel
- Do we need collocation of the OS and the kernel on the kernel-level?
- Specialization – catering needs of a service to an application
- Extensibility – adding new functionality to the OS

Abstractions

- Threads –
- Address spaces –
- Inter-Process-Communication (IPC) –

L4 Architecture



System calls:

- monolithic OS:
 - `read()` → GNU libc → syscall → trap → kernel
- L4:
 - App → Linux → L4 → libc → `read()` → return → Linux → App
 - extra messages (IPC) in L4 adds overhead
 - interrupts, exceptions, page faults → all handled by messages

Memory Management

- Hierarchical address spaces
- Linux delegates memory to apps
- L4 grants memory to Linux
- user-level pages/pagers
 - page-fault → traps to μ Kernel → Linux → update page-tables etc.

Page Tables

- 2 sets of page tables
 - one at user-level stored with Linux
 - one with the μ Kernel – shadow page table
- Concern arises when updating the tables
 - overhead when updating (double update required)
 - extra memory spent for duplicate copies

L4 Linux Implementation

- modify libraries (eg. syscalls) → GNU libc, libc.a
- modify architectural dependent portions of linux
 - page-fault → linux tries to update hardware TLB ; instead instruct moves to L4

Conclusions/Answering Goals

- 5-10% slower than monolithic OS – not big deal
- No need co-location, performance is good enough
- Specialization → pipes for better performance ← achieved through memory mapping

- extensibility → cache partitioning ← through software management
- user-level-pagers

L4 Summary

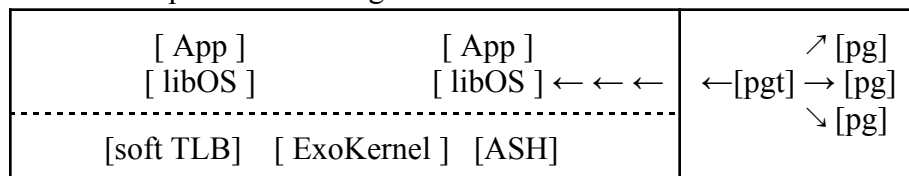
- Design of a modern microkernel
- prefer approaches to monolithic kernels
- Opportunities of specialization and extensions.

D. R. Engler, M. F. Kaashoek, and J. O'Toole Jr., [Exokernel: An Operating System Architecture for Application-Level Resource Management](#), In *Proceedings of the Fifteenth ACM Symposium on Operating Systems Principles (SOSP'95)*, December 1995.

Q: Compare and contrast an exokernel with a microkernel.

Exokernel Goals:

- Security
- Efficiency
- Expose hardware to user-level
- Push as much functionality of OS to user-level as possible
- Maximize what processes manage themselves



Systemcalls + Memory Management

- Benefits
 - lower overhead: read() → libOS → syscall → return
 - easy to specialize + extend
 - User-level management of hardware resources
 - protection via *secure bindings*
 - PTE - check before installing in TLB
 - tables - physical pages per process
 - decouple from authorization for usage
 - *resource revocation & abort protocol*
 - app asks libOS for a physical page
 - track resource ownership
 - ASH – handles packet processing ; modern day → eBPF (kernel modification for improved network performance)

Adoption

- Inspired approaches to virtual machines
- high-performance settings → datacenters, cloud servers

ExoKernel Summary

- Untrusted library operating system → want to manage hardware resources directly
- protection using *secure bindings*

Virtualization - 10/28

R. J. Creasy, [The Origin of the VM/370 Time-Sharing System](#), In *IBM Journal of Research and Development*, September 1981.

Q: What was VM/370's motivation for virtualizing the hardware?

Type 1 Virtual Machine: (Bare Metal)	[ControlVM] [VM] ... [VM] [syscalls^] [GuestOS] [GuestOS]
Xen, VMWare Server, HyperV, VM/370	Hypervisor
Type 2 Virtual Machine: (Hosted Model)	[Linux] ... [GuestOS] [VMWare] [VirtualBox]
VMWare, VirtualBox	Windows/MacOS/Linux

VM 370

IBM did virtualization first, in the 1960s before everyone else, but their code was not open source so it did not catch on with the general public

Goals / Why VMs?:

- OS Development - much easier to experiment and develop user-level programs
- Isolation - protection between aspects of the system
- Alternative to batch processing
- Management of multiprocessing
- Emulating new hardware
- Continuous operation - trying new OS could be done by just switching processes

Structure

- Control Program (CP) – hypervisor, runs at privilege
 - Could run CP in user-level to test; would need another CMS layer on top
- Conversational Monitor System (CMS) – guestOS, runs at user level
 - used physical memory, errors would lead to system crashes
- Remote Spooling and Communication System (RSCS) – network connection

Virtualization in CM/370

- challenge: handling privileged instructions (ex. halt(CPU), modifying TLB, etc.)
 - trap from CMS to CP, emulates OS behavior and reallocates CPU
 - ^ trap and emulate

P. Barham, B. Dragovic, K. Fraser, S. Hand, T. Harris, A. Ho, R. Neugebauer, I. Pratt, and A. Warfield, [Xen and the Art of Virtualization](#), In *Proceedings of the 19th Symposium on Operating System Principles (SOSP'03)*, October, 2003.

Q: Microkernels and virtual machine monitors are two different ways to support the execution of multiple operating systems on modern hardware. How does the microkernel approach in L4 compare and contrast with the VMM approach in Xen?

Xen and the Art of Virtualization

Influential

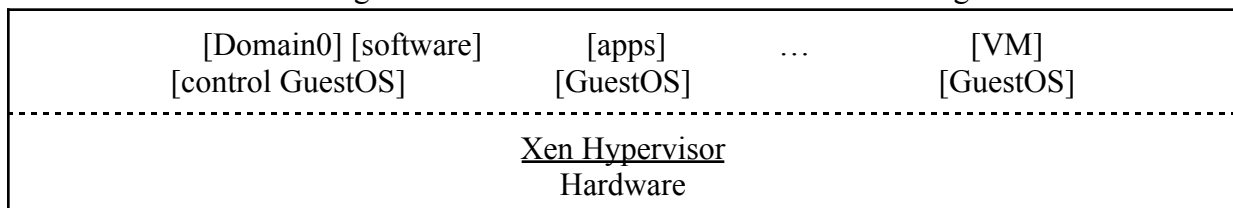
- released open source → made it very popular and easy access
- academia → OS research and experimentation becomes easier
- industry → Xen becomes the framework that **Amazon AWS** gets built on

Goals / Why VMs (in the 2000s)?

- VMWare - similar motivations as VM/370
- Xen – datacenters, cloud computing was on the rise but still very early
 - Goal: rent out computing power from servers
 - Allows user to set up their own system/machine configuration, without needing to worry about any hardware

Requirements for datacenter VMs:

- Isolation – **security** between users, expected **performance** being paid for
- Scalability – number of users per machine
- Minimal overhead – users should feel as close to bare metal as possible
- Heterogeneous OS – users can bring any OS of their choice
- Resource accounting – know how much resources are used to charge users



Challenges of virtualizing x86 architecture:

- Hardware managed TLB (x86 and ARM) →
 - software managed TLB allows hypervisor to control what gets replaced on a fault
 - hardware managed TLB has the hardware walk the page table to replace the entry
 - No hypervisor intervention to choose TLB entries

- Trap and emulate does not always apply in x86, not all syscalls lead to a trap, so the OS doesn't respond consistently to every privileged instruction.
 - `popf` instruction → pops value from stack and fills registers, supposed to fill privileged and non-privileged registers, but running at user-level only fills the non-privileged registers.
- I/O overheads, VMs should not have physical access to I/O

Xen's Solution: *paravirtualization*

- paravirtualization – modifies the guestOS to cooperate better with the hypervisor
- hypercalls – replaced privileged instructions with syscalls into Xen
- developer modifies guestOS to deal with privilege

VMWare's Approach: binary rewriting

- Wanted to support unmodified OS, even when you don't have source code.
- scans blocks of upcoming instructions, if it finds a privileged instruction it calls hypervisor as needed by guestOS
 - adds overhead, can be optimized (ex. caching)
- Software modifies guestOS to deal with privilege

CPU

- Key approach: paravirtualization
- syscalls, exceptions, interrupts
 - Xen routes to guestOS
 - **syscalls** → pre-validates syscall handler, allowing trap to hypervisor bypass

Memory

virtual	"physical"	hardware/physical
guest OS page table		Xen
uncooperative guest OS → page tables ↗		
————— → page tables ↗		

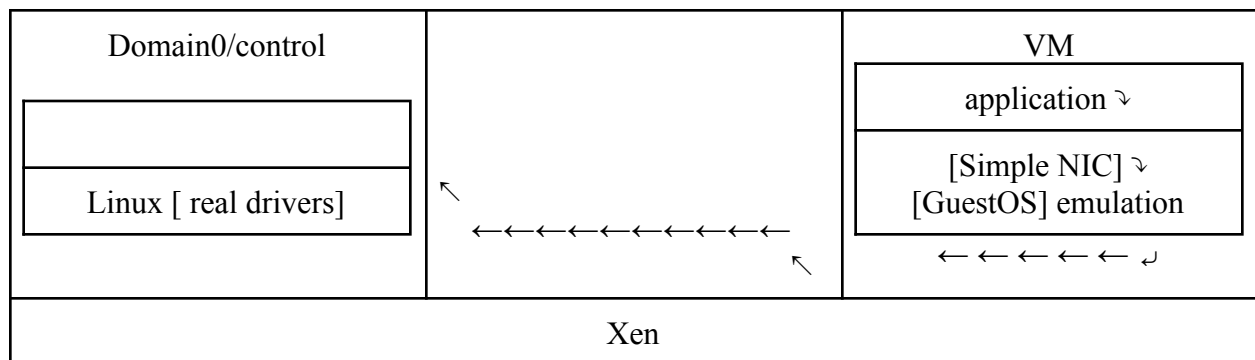
- page tables – read only for guestOS
- Xen validates and updates physical PTEs
- Similar to exokernel where the guestOS updates PTEs and hypervisor updates physical

VMWare

virtual	shadow page tables	hardware/physical
guestOS PTEs	page fault = MMU update →	TLB
page tables ↗		
page tables ↗		

I/O

- Disk, network
- challenge: many different drivers for peripherals; would require too much modification



Hardware support for virtualization

CPU

- ring3 – applications ↘ trap
- ring0 – guestOS ↘ trap
- root ring0 – hypervisor

Memory

- Intel's Extended Page Tables (EPT)
 - Hardware can walk both sets of page tables ("physical" and hardware)

I/O

- Steering I/O –
 - Single Root I/O Virtualization (**SR-IOV**) – expose virtual slices of a network card, disk etc.
 - Each virtual machine thinks they have the whole peripheral

- manage memory

Virtualization Summary

- trap and emulate
- x86 is hard to virtualize
 - paravirtualization
 - binary translation
 - hardware extension

Virtualization - 10/30

S. Soltesz, H. Pötzl, M. E. Fiuczynski, A. Bavier, Larry Peterson, [Container-based Operating System Virtualization: A Scalable, High-performance Alternative to Hypervisors](#), In *Proceedings of the 2nd ACM SIGOPS/EuroSys European Conference on Computer Systems (Eurosys'07)*, March 2007.

Q: What are the key tradeoffs between hardware virtualization and OS-level virtualization?

Hardware virtualization provides better isolation between systems and processes but incurs more overhead due to hypervisor management and operating system variety. In contrast, OS-level virtualization allows for faster resource usage and incurs less overhead for OS management because the system relies on a singular shared kernel. As such, OS-level has less isolation and variety but benefits from more efficient resource usage and less overhead.

1. The paper addresses the need for virtualization methods that provide high degrees of isolation and efficiency for usage in “HPC clusters, Grid computing, hosting centers, and distributed platforms like PlanetLab”. (Abstract) While hypervisors like Xen and VMware provide strong isolation, they come at the cost of efficiency when running many VMs with similar OS distributions.
2. This paper’s key difference from previous papers is that new container-based systems isolate VMs at the syscall layer rather than at the hardware abstraction layer like hypervisors (2.2). This approach shares a single kernel across all VMs, which reduces overhead but requires more careful implementation of security.
3. Security is implemented through “separation of name spaces (contexts) and access controls (filters)” (3.1). Contexts separate the namespaces of containers so each process is isolated to their users through UID/GID, processes through PIDs. Network separation is completed by a networking subsystem shared between all the VMs, that only allows specific sockets to bind to sets of available ip addresses defined at VM creation or by the host VM. (3.3.2).
4. The authors justify their arguments by using external benchmarks to verify their findings for each of the subsystems they implemented. The OS syscall virtualizing was benchmarked using *lmbench* (4.2), the network subsystem was benchmarked using Iperf (4.3.1), and the file system was benchmarked using DBench & Postmark (4.3.2).
5. Overall, in my opinion, the paper was successful in describing a scalable alternative to hardware level hypervisors in the form of container-based operating systems. The authors clearly explain why their actions were different from VMware and Xen, and explain the use cases for which the Linux-VServer is a superior option.

Z., Ziqiao, Y. Shan, W. Cui, X. Ge, M. Peinado, and A. Baumann, [Core slicing: closing the gap between leaky confidential VMs and bare-metal cloud](#), In *Proceedings of the 17th USENIX Symposium on Operating Systems Design and Implementation (OSDI'23)*, July 2023.

Q: How does core slicing differ from traditional virtualization (e.g., in Xen)?

Core slicing provides hardware-enforced isolation that looks similar to bare-metal machines' security and performance. Moreover, slices of the physical resources (CPU cores, memory, I/O) are physically partitioned by the slice manager and allocated by guest users of the slice (users of the cloud machine). Traditional virtualizations are managed by a hypervisor that allocates resources to each subsystem, sharing physical resources between each of the guest OS's; meaning the guestOS's are only separated on a software level.

1. The paper describes a new architecture for container isolation called core slicing to partition hardware resources into *slices* controlled by each guest, setting isolation and security properties and removing the reliance on the hypervisor for resource allocation.
2. Previous methods had entrusted the cloud providers' hypervisor with resource management, but to promote security, new systems move toward minimizing the cloud provider's access to critical information. The paper is one more step toward that goal, moving physical resource management away from the cloud hypervisor to the slice manager that securely allocates and manages resources for dedicated "control slices".
3. A trusted slice manager (*slicevisor*) is set up in a privileged control slice, *slice0*, that allocates portions of the physical resources for each guest trying to access the machine (3.1). **Core slicing ensures security by preventing slices from interacting with each other**, as they are unable to access memory, other CPU cores, and I/O devices from other slices. Two hardware features enforce isolation: **lockable filter registers for memory and device access and secure core-local reset to reconfigure access to core slices**. Previously, the list of trusted components was a much larger set (hardware, hypervisor, management software/OS), but after core slicing, only the hardware and slicevisor during setup are trusted components.
4. The authors of the paper justify their conclusions through detailed analysis of microbenchmarks of two implementations: an x86 prototype and RISC-V prototype, demonstrating high performance metrics relative to traditional VMs (5.0). Moreover, the paper highlights the various security flaws that their new architecture is able to counter: cache side-channel attacks, and transient execution attacks (6.2).
5. The paper effectively answers their question, demonstrating the benefits and drawbacks of a slice based replacement for separation of resources on a cloud based machine.

Scheduling - 11/4

T. E. Anderson, B. N. Bershad, E. D. Lazowska, H. M. Levy, [Scheduler Activations: Effective Kernel Support for the User-level Management of Parallelism](#), In *Proceedings of the 13th ACM Symposium on Operating Systems Principles (SOSP'91)*, Sept. 1991.

Q: The goal of scheduler activations is to have the benefits of both user and kernel threads without their limitations. What are the limitations of user and kernel threads, and what are the benefits that scheduler activations provide?

User-level threads are much faster than kernel-level threads because they have no overhead of requesting permission from the kernel to function.

The authors designed a system to **track** what is **running on a core**, and allow the **user-level scheduler** to determine what threads should be running based on **needs of processes and requests** from kernel events (interrupts and exceptions).

Kernel-level threads

- OS is aware of them and manages them
 - Does not mean they only live in kernel space!
- Purpose: leveraging **physical parallelism**
- example: pthreads –

User-level threads

- Purpose: express **concurrency** in applications
- Managed by user-level threading libraries
 - Example: Java threads, Go routine, Rust async functions

[u-thread] [u-thread] [u-thread] ... [u-thread]	user
[pthread] [pthread] [pthread] ... [pthread]	kernel

Threading models

- M:N threading – M user-level threads to N kernel-level threads
 - lib creates a queue for each of the N threads, multiplexing user-threads into them
 - Usually, $M \gg N$, many user-threads, few kernel threads
 - $M = 24$ and $N = 16$, with 8 cores,
3 user-threads can share 2 cores
- M:M threading – 1:1 threads; no extra functionality from user-level library
- M:1 threading – no physical parallelism; outdated

Tradeoffs

User-level threads	Kernel-level threads
- lower overhead (fast as prod calls) - kernel events are hidden to user-level - threads block for I/O, page faults, etc - OS can preempt kernel-threads - customization to the application needs	- higher overhead cost (for everything) - have to be general purpose - less optimized for specific use cases - hardware parallelism (multicore processing) - more isolation

Scheduler Activations

[u1-thread] → read() ↘	...	[u-thread] [u-thread]	user
tell user blocked → [cpu ₁ SA ₁]	[cpu ₄ SA ₄]	[cpu ₂ SA ₂] [cpu ₃ SA ₃] upcall ↗	kernel

- OS just allocates the cores to processes
 - user-level schedules threads across cores
1. u1-thread tries to read() from I/O, syscall gets sent, CPU₁ core gets blocked
 - a. library would not know this, u1-thread is just blocked now
 - b. SA₁ also gets blocked so kernel preempts other core to recover the block
 2. OS adds a core to an app: create a new schedule activation
 - a. SA₄ gets activated and CPU₁ sent as an upcall to u1-thread
 3. OS revokes a core: preempts 2 SAs, using one for an upcall

Summary

- pros and cons of user-level and kernel-level threading
- Use of upcalls to delegate scheduling to user-level library (via scheduler activations)

J. Lozi, B. Lepers, J. Funston, F. Gaud, V. Quéma, and A. Fedorova, [The Linux Scheduler: a Decade of Wasted Cores](#). In *Proceedings of the Eleventh European Conference on Computer Systems (Eurosys'16)*, 2016.

Q: What makes multicore scheduling more challenging than single-core scheduling?

Decade of Wasted Cores

Linus Torvalds

- short temper, big ego, strong personality
- designer of Git and Linux
- advocate for monolithic kernels
- advocate for open source software

Scheduler Goals

- fair allocation of compute resources
 - prioritize important tasks / enforce specific policies
- work conserving – no core is idle if there are runnable threads
- efficiency – minimize wasted time / busy waiting
- performance objects – ex. cache locality
- scalability for multicore machines
- (for Linux) general purpose

Single-Core Scheduling

- Maintain one runqueue, managing processes using FIFO order

CFS: Completely Fair Scheduler

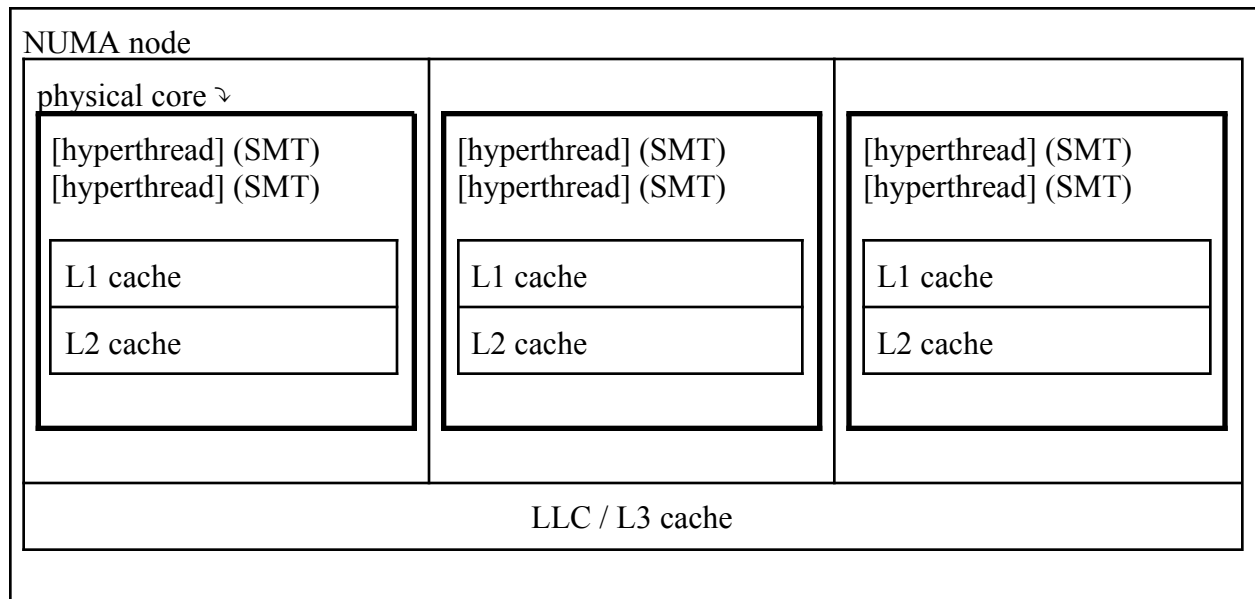
- preemptive – timeslice allows for running for quantas at a time
- weighted priority – *vruntime* labels amount of time thread is allowed to run for
 - red-black tree → priority queue implemented with a balanced tree

Multi-Core Scheduling

- one runqueue per core
 - only 1 runqueue would create overhead bc of locking management

Challenges:

- runqueue load imbalance – expensive to remedy
- Complex cache hierarchies – cache may update later than context switch per core



Load Balancing in Linux

- Periodic load balancing
- “emergency” local balancing – idle core, try to steal
- on creation/wakeup
 - try to place thread on an idle core or local core

Metrics to consider

- number of threads
 - Does not account for priority of threads
- sum of the weights
 - Does not account for blocking
- load – combines weights and average CPU utilization
- groups – threads from the same process that share data

Load Balancing Algorithm

- if we didn’t care about cache locality, we would balance all threads equally
- hierarchical approach
 - socket →
 - cores →
 - hyperthreads
- Balance lowest level first then balance going up the tree
 - hyperthreads pairs balanced first
 - then balance across all the cores in one socket
 - lastly, balance across all sockets on machine

Group Imbalance Bug

- 8 NUMA nodes, 8 cores each
- 64 compilation threads
 - load = 1
- 2 R threads
 - load = 64
- authors observed an issue regarding priorities
 - R threads received a whole NUMA node each,
 - but only used 1 core per thread, leaving 7 unused cores
 - So 64 compilation threads were forced into 6 NUMA nodes
 - 7 + 7 cores unused
- fix: balance load based on min rather than average

Overload on Wakeup Bug

- threads woke up on the local core for cache affinity, but local core was busy
- fix: prioritize waking up to idle cores; instead of local cores

Summary

- Design of a scalable CPU scheduler – per-core runqueue
- Approaches to balancing loads
- CPU scheduling is more complex than expected

Scalability - 11/6

P. E. McKenney, J. Fernandes, and S. Boyd-Wickizer, [RCU Usage In the Linux Kernel: Eighteen Years Later](#). ACM SIGOPS Operating Systems Review, August 2020.

Q: In what settings is RCU a particularly good choice, or a particularly bad choice?

RCU usage patterns

- Wait for completion
 - Waiters use *synchronize_rcu* or *call_rcu (async)* to delimit activities RCU read-side critical sections
- Reference counting
 - Leverages linux network stack, using UDP message passing
 - Searches radix tree with active PIDs to identify what object to update
- Type safe memory
 - type-specific memory stays the same type even when deallocated, if there are still references to it in memory
- Publish-subscribe
- Read-write locking
- RCU-mediate fast paths

Design patterns

- Impose level of indirection
- Mark obsolete objects
- Retry readers

Read-Copy-Update (RCU)

- Multicore CPUs
- year 2000 → multicore CPUs started popularizing
- Today → hundreds of cores
- goal: scalability, i.e., performance scales linearly with number of cores (not really tho lol)

App ₂ {[Core ₁]~ [Core ₂]~}	...	App _N {[Core ₁] [Core ₂]~}	user
syscall	syscall	OS	syscall
			kernel

- initially: synchronization imposed using one big kernel lock
- next: fine-grained locking

RCU Goals

- Concurrent reads, even during updates
- Low overhead – memory, execution time

- Deterministic completion time
 - esp during “critical interrupt handlers” like physical issues or memory failures

Example: Linked List

[head] → [cat] → [dog] → [fish] → x

- approach #1: spin-lock
 - simplest approach
 - all threads acquire and release
- approach #2: read-write lock
 - multiple readers OR 1 writer
 - allows for concurrent reads – good for high rate of readers
 - drawbacks:
 - can not read while a write is ongoing
 - could cause starvation
 - (if readers are continuously allowed, writers would be starved)
 - execution-time overhead (even w/o writers)
 - struct rw_lock {int n;}
 - n = 0 → unlocked
 - n = -1 → 1 writer
 - n > 0 → locked by n readers
 - read_lock – increment n
 - write_lock – decrement n
 - every time n is accessed from a different core, it causes a cache miss and needs to pull value from the core that last updated it
 - to just read data, still requires many writes to n
 - also requires space overhead to store each lock’s n value
- What if readers of the linked list just skip the lock?
 - no writer: okay
 - yes writer: could see inconsistent state

Read-Copy-Update (RCU) → want fast reads

- mixture of rules, patterns, and mechanisms for code development

RCU idea #1: writers make a copy

[head] → [E₁] → [E₂] → [E₃] → x

- Update E₂:
 1. acquire lock
 2. E_{new} = alloc()
 3. E_{new}→next = E₂→next
 4. strcpy(E_{new}, “new string”)

[E_{new}] →

[E_{new}] → [E₃] → x

(add memory barrier here)

5. $E_1 \rightarrow \text{next} = E_{\text{new}}$

$[E_1] \rightarrow [E_{\text{new}}] \rightarrow [E_3] \rightarrow x$

6. release the lock

- readers see old version from steps 1-5
- after step 6, readers see the new version
- IMPORTANT: readers can not see partially complete updates
- problem: compilers & CPUs can reorder instructions

RCU idea #2: memory barriers to enforce ordering

- add **memory barrier** before step 5, **forcing all other steps to occur first**
- problem: use-after-free – writer wants to delete an element (free some memory), but a reader may still have a pointer to that memory location

RCU idea #3: grace period for writers

- writers – wait until all CPUs have gone through context switch
- readers – can't hold RCU pointers through context switches
 - invalidates all old pointers from reader perspective
 - allows freeing of writer's data without worry of readers having pointers

Linux's RCU API

- readers:
 - `rcu_read_lock` } disable interrupts
 - `rcu_read_unlock` } enable interrupts
 - `rcu_dereference(pointer)` } includes a memory barrier
- writers:
 - `synchronize_rcu` } wait for all CPUs to context switch
 - `call_rcu(callback, arg)` } async version
 - `rcu_assign_pointer(ptr_addr, ptr)` – updates pointer, includes mem barrier

RCU vs. read-write locks

- readers:
 - RCU imposes almost no overheads
 - reads can happen during writes
- writers:
 - writes can take longer with RCU

Pros vs Cons of RCU

- pros:
 - only helpful if reads >> writes
- cons:

- relies heavily on context switches
- complex
- different model of consistency (for users)
 - readers can see stale data
 - readers hold pointers to data until context switch invalidates
- only works for data structures with single committing write

Summary of RCU

- multicore CPUs – need scalable synchronization method
- RCU – readers don't wait, parallel with writers
- used widely in Linux

S. Boyd-Wickizer, A. T. Clements, Y. Mao, A. Pesterev, M. F. Kaashoek, R. Morris, and N. Zeldovich, [An Analysis of Linux Scalability to Many Cores](#), In *Proceedings of the 9th USENIX Symposium on Operating Systems Design and Implementation (OSDI'10)*, October 2010.

Q: What kinds of application behavior can cause an application to scale poorly?

Shared locks, procs moving to different cores, cache/tlb misses, global locks, rw-locks

Linux Scalability → 3 research papers

- designed new OS from scratch for scalability: Corey
- authors stepped back, considered: How scalable is Linux?
- Scalable Commutativity Rule – principles of how to scale

Scalability Bottlenecks: 3 possible culprits

- App, Kernel, Hardware
- approach:
 - review set of apps under workloads
 - find/fix bottlenecks
 - iterate until OS is not causing bottleneck

Cache Coherence

4 interconnected cores with local caches			
C1 – writes to shared variable		C2 – reads → needs to pull from C1 cache if C2 – writes → needs to invalidate C1	
C3		C4	

- cache coherence protocol
 - keeps caches consistent
 - adds overhead

Scalability Bottlenecks:

- writing to shared variables/memory
- locks on shared data – only one thread can hold lock at once
- competition for caches or memory bandwidth
- not enough concurrency
 - app only has 2 threads, can't use more than 2 cores

Techniques of Improving Scalability

- Sloppy counters – partition counters across cores
 - have cores update counters on their own core
 - only change the global on rcu_synchronize
 - true value of counter = value of global - sum(per-core counters)
 - global value ends up being a fast access upper bound for counter
- avoid lock contention
 - fine-grained locks
- avoid data contention
 - per-core data structures
- wait-free synchronization
 - RCU is an example
- avoid false sharing
 - cache lines, 64B [::thread1.data:::thread2.data::] cache line
 - fix: move shared variables to separate cache lines
 - struct {
 thread1 data;
 byte[] padding; fill cache line to geo-separate data
 thread2 data;
 }

Summary

- General techniques for scalability
- Linux is actually pretty scalable
 - Funny bc they just wrote a completely different OS to disprove this

Networking - 11/13

A. D. Birrell and B. J. Nelson, [Implementing Remote Procedure Calls](#), ACM Transactions on Computer Systems, February 1984.

Q: In what ways do the authors optimize connection management?

Pointers to contemporary RPC and serialization packages: [Google RPC](#) & [Protocol Buffers](#), [Facebook Thrift](#)

What does a *network* actually look like?

client [] NIC → [] server
 ← endhost
 switches/protocols

Remote RPC

network communication → cast as a procedure call

Goals:

- Identical semantics as local procedure calls
- Efficiency – computation time on CPUs for # of messages
- Security – ensure data is secure over open networks
- High performance – small request
- Generality – ease of use for many applications/languages

client [application]	server [application]
prod_call ↘	
messages ↘ [stub] RPCRuntime	[stub] message ↗ → RPCRuntime ↗

RPC Example

client	server	
s = add(5,7)	int add(int x, int y) { return x + y; }	app
int add (int x, int y) { put x, y into buffer send message extract result return result }	add_stub (buffer) { extract arguments s = add (x, y) put s in buffer send response }	RPC library

- Supports many data types (marshalling/serializing)

Optimizations in the RPC Runtime

- transport protocol -
 - piggyback ACKs with data (bundle acknowledgement with responses)
 - synchronous –
 - like regular prod calls, client blocks until receiving response
 - only 1 rpc allowed at a time (per client)
- processes – pool of idle server processes
 - dispatch → getting to and from processes
- connections –
 - no setup, no teardown
 - TCP uses SYN/ACK/SYN handshake

Binding

- server exports an interface, clients import interface
- grapevine
- binding at runtime

Exceptions/Errors

- Server or network can fail independently of local machine
 - How would they notify the client if they're already down.

Adoption

- Datacenter applications
 - Google: gRPC
 - Meta: Apache Thrift
- Secure enclaves
- IoT
- Browsers – HTTP
 - packaging and sending data through browser, and displaying to user

Summary

- RPC makes remote functionality appear local
- stubs, binding
- performance optimizations

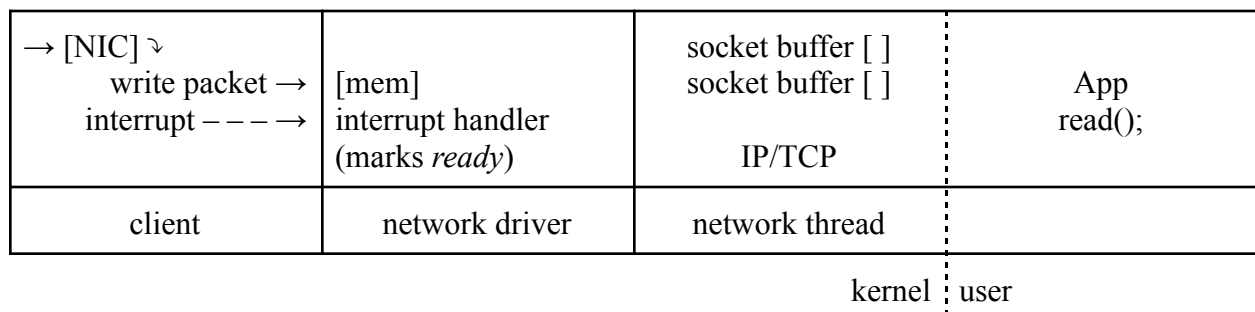
J. C. Mogul and K. K. Ramakrishnan, [Eliminating Receive Livelock in an Interrupt-driven Kernel](#), ACM Transactions on Computer Systems, August 1997.

Q: What are the tradeoffs between interrupts and polling?

Applications:

- passive network monitoring – tracking all of the statistics
- packet processing in software – routing, firewall, VPNs, load balancer
- Network services – file server, web servers, DNS

Network Stack

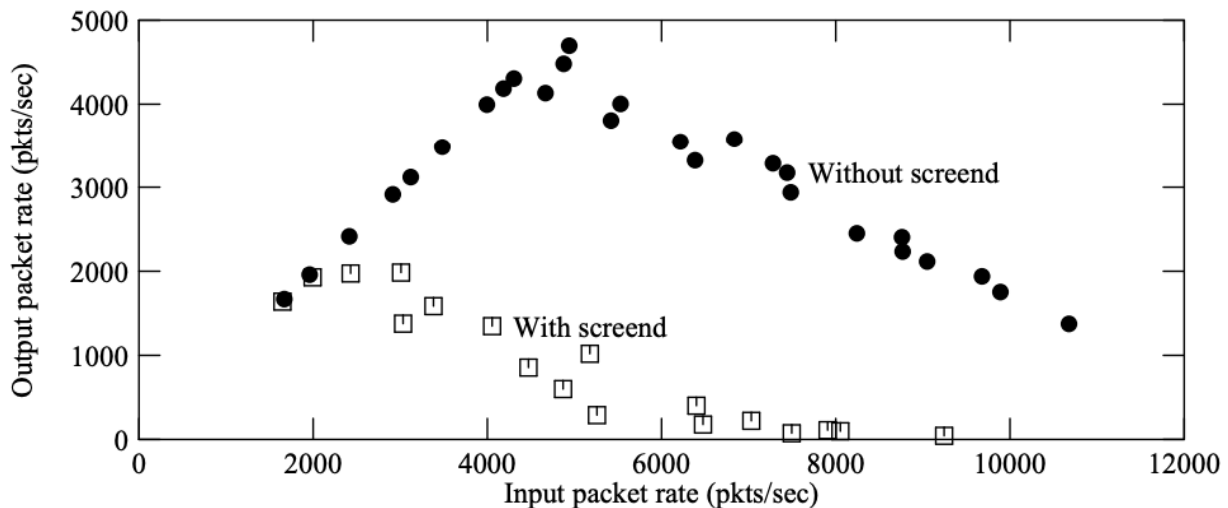


- intermediate queues absorb temporary bursts of request
- allows for variable control of different parts of stack

Receive Livelock

- overloaded → system spends all its time handling interrupts
 - causes throughput collapse

✓ MLFRR → maximum loss free receive rate



Polling – Software periodically checks if packets have arrived

Polling	Interrupts
<u>pros:</u> - control over I/O processing - fairness with round robin - efficient at high load <u>cons:</u> - can add overhead if too frequent - high latency if polling too frequent - could drop packets if too infrequent	<u>pros:</u> - work is proportional to packet arrival; - less packets = less work <u>cons:</u> - can trigger receive livelock - overhead from context switches - vulnerable to side-channel attacks

Solutions

- **hybrid approach:** use interrupts to detect first packet, polling to get receive rest
- reserve CPU time for applications, temporarily disabling input
- **drop packets early:** systems overloading, packets get dropped; drop before processing
- removed layer of queueing by combining network driver and network thread
- **round-robin** polling to allocate resources fairly between events

Summary

- introduces the concept of ‘receive livelock’
- hybrid approach to solve the presented problem
 - when system is overloaded: drop packet early as possible

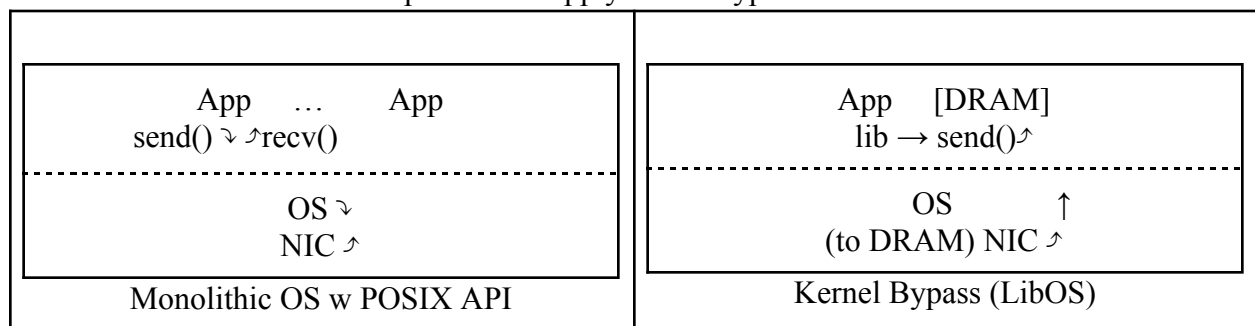
Networking - 11/18

A. Belay, G. Prekas, A. Klimovic, S. Grossman, C. Kozyrakis, and E. Bugnion [IX: A Protected Dataplane Operating System for High Throughput and Low Latency](#), In *Proceedings of the 11th USENIX Symposium on Operating Systems Design and Implementation (OSDI'14)*, October 2014.

Q: What is the difference between the control plane and the data plane and why does IX separate them?

Datacenters

- Applications must scale across many machines
- Networks have very high bandwidth, and keep increasing regularly
 - Currently: 100 - 400 Gbit/s and still going up
 - avg 1500 byte packets can arrive ~ 120 ns
 - syscall is way more expensive than 120ns;
 - so per-packet kernel calls are not viable.
- Storage: high-speed flash, NVM (non-volatile memory)
- Must minimize overheads
 - Kernel is too expensive → apply kernel bypass



dataplane vs control plane

- dataplane – sending & receiving packets
- control plane – Management of the data

challenge: isolation between apps

- Modern NIC hardware:
 - virtual queues – ex. per application
 - IOMMU – memory isolation for I/O devices
 - Allows NIC to use virtual addresses

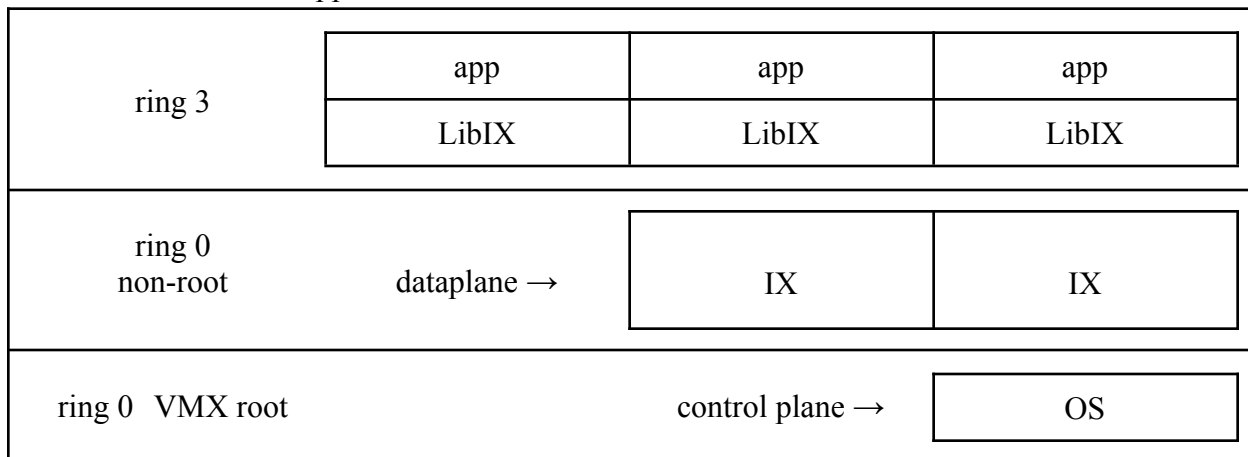
IX - exokernel approach for operating systems in data centers

- Protection – isolates between apps, and isolation for network stack
- Microsecond tail latency – large fanout, tail latency (99%)

- High packets rate – often very small packets
 - Lots of volume and many concurrent connections
- Resource efficiency – cores, memory
- other papers have tried to answer each of these before, but never all at once. LibOS was the closest, but it violated the protection rules.

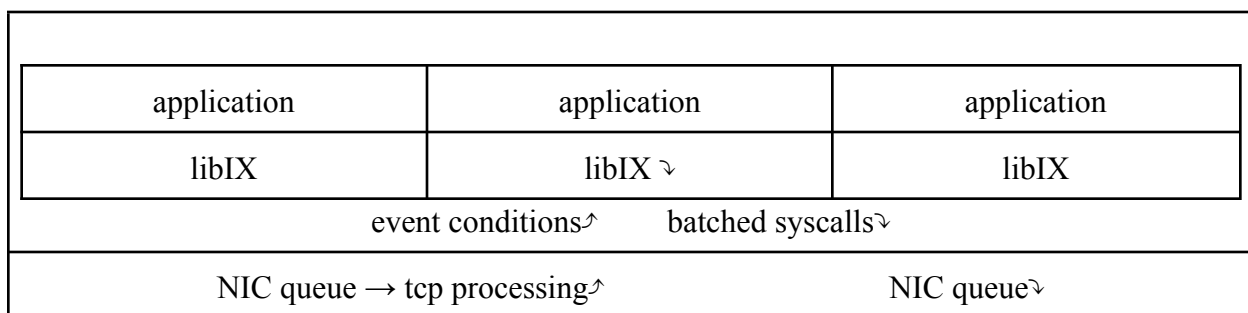
IX Overview

- Use hardware support for virtualization



run to completion with adaptive batching

- rtc - process packets from start to finish; minimize context switching (pro-cache locality)
- ab - process many packets at once, batch size varies dynamically to a maximum



synchronization – free processing

- partition resources across cores
- Separate per-core NIC queues
 - RSS (Receive-Side Scaling) – hash of packet fields to put cores from connection to the same core (for cache affinity)

Zero Copy

- POSIX – uses many copies (esp for networking)
- Zero Copy – use pointers to packets instead of copying over
 - NIC writes packet to app - accessible memory

Evaluation

- Throughput and latency – winning compared to linux
 - Avoids interrupts overhead
 - More efficient overall

Summary

- protected dataplane using virtualization
- Run-to-completion, batching, RSS, Zero Copy

M. Marty, M. de Kruijf, J. Adriaens, C. Alfeld, S. Bauer, C. Contavalli, M. Dalton, N. Dukkipati, W. C. Evans, S. Gribble, N. Kidd, R. Kononov, G. Kumar, C. Mauer, E. Musick, L. Olson, E. Rubow, M. Ryan, K. Springborn, P. Turner, V. Valancius, X. Wang and A. Vahdat, [Snap: a Microkernel Approach to Host Networking](#), In *Proceedings of the 27th ACM Symposium on Operating Systems Principles (SOSP'19)*, October 2019.

Q: What are the tradeoffs between the monolithic, libOS, and microkernel-based approaches to structuring a network stack?

Snap Goals – wanted to move network stack to user level

- Easier deployment – originally 1-2 months to deploy kernel, wanted to reduce this time
- Easier development – kernel code requires more skill and specification
- More optimization – allows more per-process optimization

App [] ← queues → [network module] App [] → queues ↓		microkernel user
OS	NIC	kernel

LibOS (Exokernel)	Snap (Microkernel)
pro: • tailored network stack • better latency • better isolation - no shared NIC cons:	pro: • centralized management ◦ better control of scheduling • ease of deployment • decouples scheduling from apps cons: • potentially higher overhead

threading - kernel - level threads

- Don't need to modify as much to use Snap, need to change network libraries

IPC - communication between apps & Snap

- apps and Snap run on different cores so they don't need interrupts to call on each other

separate of control and dataplane

CPU Scheduling

- dataplane → engines
- Dedicated Cores: Snap[], Snap[], Apps[], Apps[]
- Spreading Cores: [Snap [1/5] App [3/5]] [Snap[1/2]] [] [App]

- Comparing Cores: Snap[], Snap[½ App [½]] App[], App[]

Best efficiency ? compacting

Best tail latency ? spreading (leveraging interrupts), dedicated (if high amount of Snap cores)

Transparent Upgrades

- Startup new Snap

For each engine:

- write old Snap state to memory] brown out period
- new Snap reads state and prep engine]
- stop executing old Snap engine }
- transfer any modified state (from brownout) } blackout period
- restart engine on new Snap } ~ 250 ms

Kill old Snap

Summary

- Microkernel based design – easier to upgrade and deploy
- scheduling modes for engines
- Widely used within google's architecture (more than ½ of their machines)
- academia tends to focus on performance
 - industry focuses on other goals → like testability, ease of deployment

File Systems - 11/20

M. K. McKusick, W. N. Joy, S. J. Leffler, and R. S. Fabry, [A Fast File System for Unix](#), ACM Transactions on Computer Systems, August 1984.

Q: In FFS, reading is always at least as fast as writing. In the old UFS, writing was 50% faster. Why is this?

Live Demo Notes

- Old school harddrive, 25mb, quite heavy, two platters
- FusionIO - 200gb early ssd, used pcie, cost \$25,000
- Disk
- Platter
- Surface
- Track
- Sectors - 512 bytes Block size - 4kb (used by OS file system)
- Arm
- Head

Goals:

- Performance – reading a file used to require **milliseconds** of seek
 - Minimize seek is the goal for file systems
- Reliability –
 - Superblock is first block read on filesystem, critical for fs to function
 - Superblock is replicated across the disk to ensure file system integrity

old file system blocks:

inodes - beginning of the disk
data

Problems:

- random block placement – cylinder groups
- inode placement – each cylinder group gets a copy of the inode
- low b/w util – only 5% of file access time was reading
 - quadrupling the block size (1024 → 4096 bytes) improved util greatly
 - increased max file size to 4gb by allowing more inodes in each block
- small max file size – using larger blocks causes internal fragmentation

Old → writes were faster bc reads took hella seeks on spread out data

New → reads got faster, but writes require more overhead (allocated inodes, update blocks etc)

M. Rosenblum and J. K. Ousterhout, [The Design and Implementation of a Log-Structured File System](#), Proceedings of the 13th ACM Symposium on Operating Systems Principles (SOSP'91), December 1991.

Q: When we want to read a block in LFS on disk, how do we figure out where it is?

Trends

- CPU ↑ Memory ↑ → FBC ↑ Disks Latency ~ B/W ↑
- write dominated IO cache
- Small files are the bane of efficiency for file systems

FFS is a **modify-in-place** file system

data blocks don't get moved around, they just get changed as needed

There's always a gotcha with systems papers. LFS's is the overhead of cleaning the logs.

The attempted to sort and split using hot-and-cold policy to clean two groups of files based on the utilization of the files in the group – with the idea of promoting simple locality

- Hot = 10% of the files that are selected 90% of the time
- Cold = 90% of the files, that are only selected 10% of the time

Turned out this was worse than just random cleaning, more locality led to worse performance.

Data structure	Purpose	Location	Section
Inode	Locates blocks of file, holds protection bits, modify time, etc.	Log	3.1
Inode map	Locates position of inode in log, holds time of last access plus version number.	Log	3.1
Indirect block	Locates blocks of large files.	Log	3.1
Segment summary	Identifies contents of segment (file number and offset for each block).	Log	3.2
Segment usage table	Counts live bytes still left in segments, stores last write time for data in segments.	Log	3.6
Superblock	Holds static configuration information such as number of segments and segment size.	Fixed	None
Checkpoint region	Locates blocks of inode map and segment usage table, identifies last checkpoint in log.	Fixed	4.1
Directory change log	Records directory operations to maintain consistency of reference counts in inodes.	Log	4.2

File Systems - 12/2

G. R. Ganger, M. McKusick, C. Soules, and Y. Patt. [Soft Updates: A Solution to the Metadata Update Problem in File Systems](#), ACM Transactions on Computer Systems, May 2000.

Q: What is the metadata update problem?

Optional recent paper about a variant of Soft Updates that uses the Rust compiler:

H. LeBlanc, N. Taylor, J. Bornholt, and V. Chidambaram, [SquirrelFS: Using the Rust Compiler to Check File-System Crash Consistency](#), In *Proceedings of the Eighteenth USENIX Symposium on Operating System Design and Implementation (OSDI'24)*, July 2024.

Soft Updates

Track when an inode's metadata is updated using dirty bits, keeping the file on the buffer cache until it needs to be evicted, only move to disk as needed, must check if inode is complete before updating directory block with {file:inode}, else revert the updated directory.

Problem: File System consistency after a crash

FS inconsistencies

- Lost file data – tolerable for performance tradeoff
- Inconsistent metadata – not okay bc of undefined behavior
 - metadata:
 - inode
 - directories
 - free-list bitmap

`fsck` – file system consistency checker

- Traces all pointers in file system, checks inode state matches expected pointers in directory, checks that inodes in use are flagged on bitmap, etc.
- Can't always fully restore consistency
- Takes a logn time to run

File buffer cache – caches file data

- Write to storage periodically (ex. ~30 sec)
- Used for **both** data and metadata
- `fsync` – force everything to get flushed to disk

Soft Updates Goals

- Ordering constraints:
 - No dangling pointers

- No reuse until older pointers removed
- Never reset an older pointer until the new one persists

main approach:

- Write changes to file buffer cache
- Track dependances between blocks
- Write blocks in a safe order

Example:

- create a file:
 - create an inode → first
 - add directory entry → second
- delete a file:
 - re-initialize inode → second
 - remove directory entry → first

	inodes
2	metadata
1	metadata

directory		
a.txt	inode = 7	
b.txt	Inode = 3	1
c.txt	Inode = 1	
...	Inode = ...	2

- fine-grained dependency tracking
 - track inodes, directory entries, etc. rather than blocks
- undo / redo
 - temporarily undo operations

Overheads

- Memory OH – storing dependency info
 - Bounded by 30 second limited writes
- CPU OH – undo / redo work
 - Not too bad for HDD bc file writes take so much compute already

Optimizations

- write many blocks at once
- skip writes that are undone
 - temporary files in /temp

Evaluations

- FFS – “Conventional” synchronous metadata writes
 - reliable
 - poor performance
- FFS – “No order” async writes
 - best performance
 - no reliability
- Soft Updates – within 5% of “no order”
 - good performance
 - and reliable
- Write-Ahead Logging (aka journaling)
 - Good performance
 - Reliable
 - More overhead for cleaning logs

Summary

- metadata consistency & ordering constraints
- contributions:
 - fine-grained dependency tracking & rollback (undo/redo)

R. Kadekodi, S. Lee, S. Kashyap, T. Kim, A. Kolli, V. Chidambaram, [SplitFS: Reducing Software Overhead in File Systems for Persistent Memory](#), In *Proceedings of the 27th ACM Symposium on Operating Systems Principles (SOSP'19)*, October 2019.

Q: Why does it make sense to design file systems for persistent memory differently?

SplitFS

Persistent Memory

- similar to RAM but protected from shutdowns
- Attached to memory bus – like DRAM
 - versus: Disk & SSD attached to PCIe
- Properties
 - Capacity – more than DRAM, less than disk
 - Latency – 2-4x slower than DRAM
 - Expensive
 - Use load/store to access – no need to request I/O like disk
 - Non-volatile (crash consistent)
 - Limited lifespan
 - Byte addressable (no need to read by blocks)
 - Bandwidth – $\frac{1}{2}$ - $\frac{1}{3}$ of DRAM

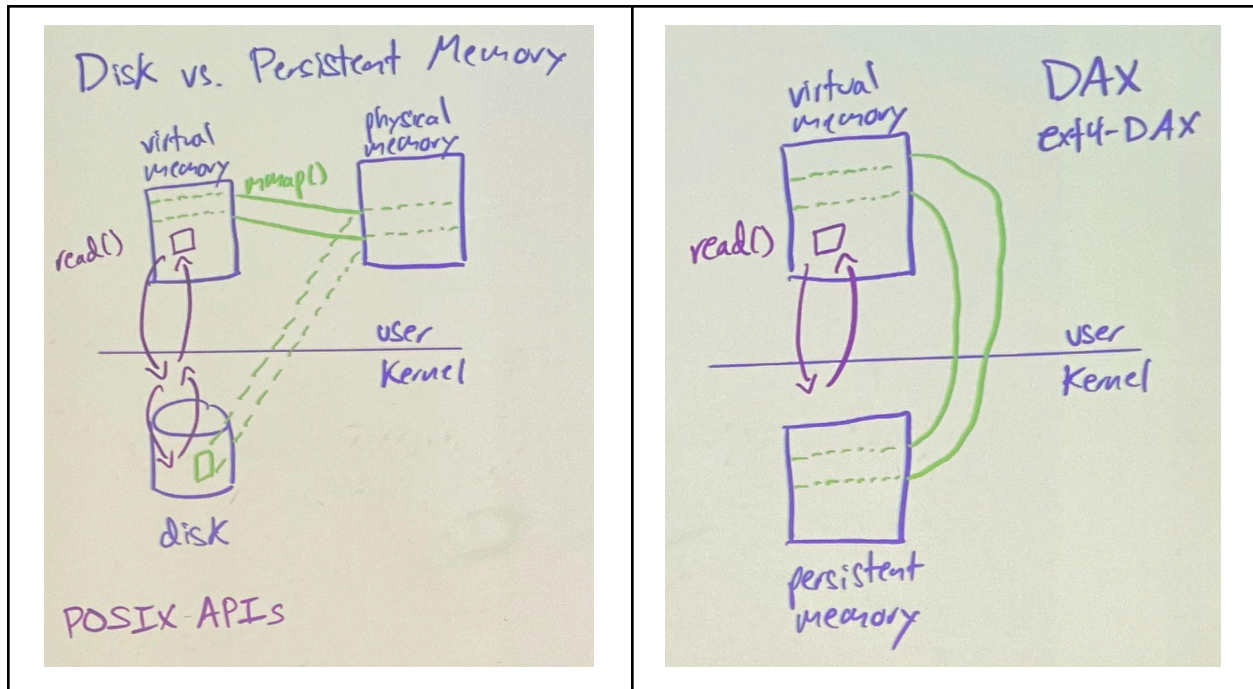
Challenge: caching

- data written only to cache is useable but not persistent; must ensure data is written to PM
- Need to use write-through stores
- Non-temporal store: MOVNT on x86
- flush: CLFLUSH on x86
 - must use fences (mem-barriers) to prevent compiler reordering the flush

SplitFS

- Past approaches: in-kernel, in userspace, hybrid
- Data – handled in userspace
- Metadata – handled in kernel
 - Goal: unmodified apps

Disk	Persistent Memory
------	-------------------



- Writes
 - Overwrites: replace content, same idea as reading through buffer
 - appends: lots of metadata to update – expensive OH
 - staging file: file in user-space; mapped to PM
 - First write to staging file
 - Second push to PM when closing file or fsync
- Relink
 - inode in staging file with pointers to data blocks
 - instead of copying from StagingFile to disk,
 - add pointers to inode in actual file
 - nullify pointers in the staging file
 - still requires trap() to OS, but skips the copying

Summary

- Hybrid user/kernel to optimize for data operations while trapping to OS for metadata
 - Staging files
 - Relink operation

File Systems - 12/4

S. Ghemawat, H. Gobioff, and S. Leung, [The Google File System](#), In *Proceedings of the Nineteenth ACM Symposium on Operating Systems Principles (SOSP'03)*, October 2003.

Q: From an application's perspective, how is using GFS different from using a local file system?

Optional paper that applies verification to file systems: H. Sigurbjarnarson, J. Bornholt, E. Torlak, and X. Wang, [Push-Button Verification of File Systems via Crash Refinement](#), In *Proceedings of the 12th Symposium on Operating Systems Design and Implementation (OSDI'16)*, November 2016.

Google File System (GFS) – user-level file system

- 2003
- inexpensive hardware -> failures are common
- very large files - 100 MB - 100 GB
 - small ish # of them
- writes: sequential append heavy
- clients are within Google
- high throughput rather than low latency
- concurrent writes to the same file

Approach:

- Files
 - large chunks – 64 MB each
 - stored as regular Linux files
- Namespace
 - hierarchical ex: a/b/c/foo.txt
- Directories
 - Kind of an illusion a/b/ != directory

GFS Architecture

- Master
 - Control operations
 - garbage collection, file mapping
 - Stores metadata
- Client
 - user/app accessing files – API to access chunks
- Chunk servers
 - Stores file data
 - Replicate each chunk

Read access flow:

1. client -- → master ask for chunkserver holding data
 a. client ← master return list of chunkservers
2. client → chunkserver choose closest chunkserver and request
 a. client ← chunkserver return data to client

Write access flow:

1. client -- → master request to write data
 a. client ← master return nearby servers on different machines
2. client → p_chunkserver choose primary_chunkserver and send data
3. p_chunkserver → chunkserver(s) aggregate data to other distributed servers
 a. chunkserver → p_chunkserver respond once data is distributed
4. p_chunkserver → client notify client to commit data to chunkservers

- Chunkservers and clients don't cache data directly,
 - most of the data is on streams, data not reassessed often
 - but linux may cache on its own level below GFS.
- Master holds metadata with version numbers to ensure all clients receive updated data.

challenge: concurrent appends

- POSIX: write(fd, offset);
- If multiple writers are trying to write to the end,
 - each think they own the EOF chunk
 - clients overwrite each other
 - Synchronization would be very expensive
- Solution → new interface:
 - record append:
 - primary chunkserver orders appends
 - At-least-once semantics:
 - Ensures all chunks occur at least once,
 - crashed chunks may duplicate

Exposing inconsistent data to clients

- Duplicate files from crashes
 - assign recordID to check if chunk has already been read
- Padding between chunks in the server
- Fragmentation records from partial writes
 - checksum to see if total data accessed for both

- Why is it okay?
 - System is much faster without checks
 - Google engineers using GFS would be able to understand the issues and work around it
 - Hardware is cheap and large scale so inconsistencies are bound to happen
 - Easier to adjust for it than try to stop it
- What happened after?
 - GFS got expanded to more than just engineers, so inconsistency became more annoying
 - Google introduced Colossus to replace GFS in 2010, fixed inconsistencies

Summary

- Distributed file system
- Different from other FS:
 - exposes inconsistencies to applications
- Challenges:
 - Scale
 - Consistency
 - Failures
-